

Advanced Natural Language Processing Pre-Training and Fine- Tuning Large-Language Models

Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes and Dr. Aki Harma – Course year 2026-2027



Maastricht University

Learning Goals Today

- Understand the basics of Pre-Training BERT, GPT-2 and T-5 from scratch
- Understand how to create better Word Embeddings with BERT
- Understand the basics of Fine-Tuning BERT, GPT and T-5
- Alignment of Generative AI Models (aka staying withing human acceptable boundaries and “reasoning”)
- Understand techniques such as Distillation, Pruning & Quantization to lower energy consumption and the risks of adversarial attacks.

How About RAG?

- In this lecture we do not discuss Retrieval Augmented Generation (RAG) and it's aspects such as:
 - Training for neural re-ranking in search
 - Aligning embedding spaces with different dimensionalities
 - Merging embeddings in encoding, cross-attention and decoding

These are all discussed in the course
Information Retrieval and Text-Mining (IRTM).

Part of the Implementation:

Data Set Creation for NLP

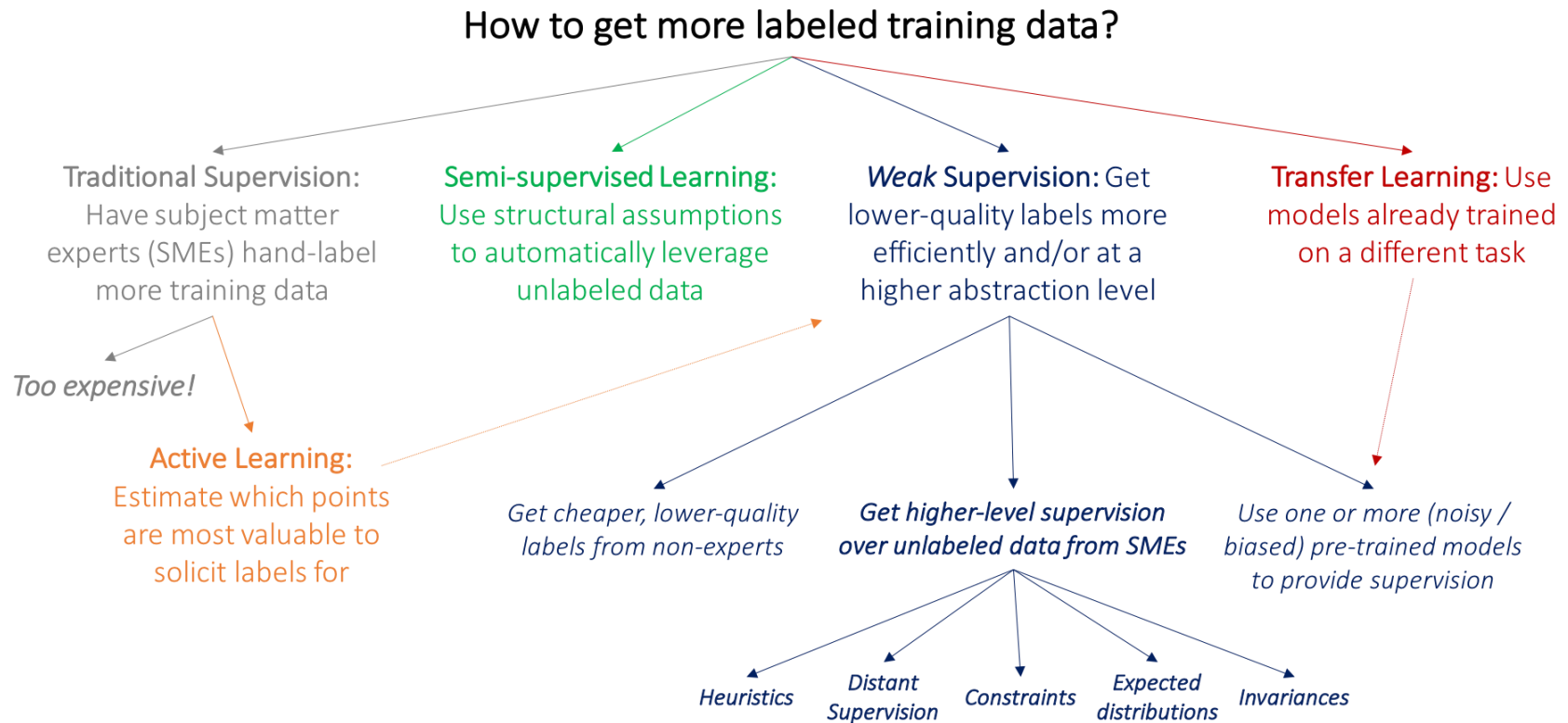


Maastricht University

Data Sets for Fine Tuning?

- Deep learning is data hungry: so, it is essential to have sufficient training data.
- There is always a need for (human) annotated or ranked data sets. Definitely for Encoder and Encoder-Decoder models, but for decoder-only models.
- There are many methods to generate additional training data (augmentation, etc).
- This is a very active field of research.

Methods to obtain more Labeled Training Data



Source: <http://ai.stanford.edu/blog/weak-supervision/>

Training Transformers

Encoder-only models
Decoder-only models
Encoder-Decoder models

Training from scratch (aka pre-training)
Fine-tuning for a particular task
Alignment of LLM's



Maastricht University

Remember Back-propagation: Stochastic Gradient Descent

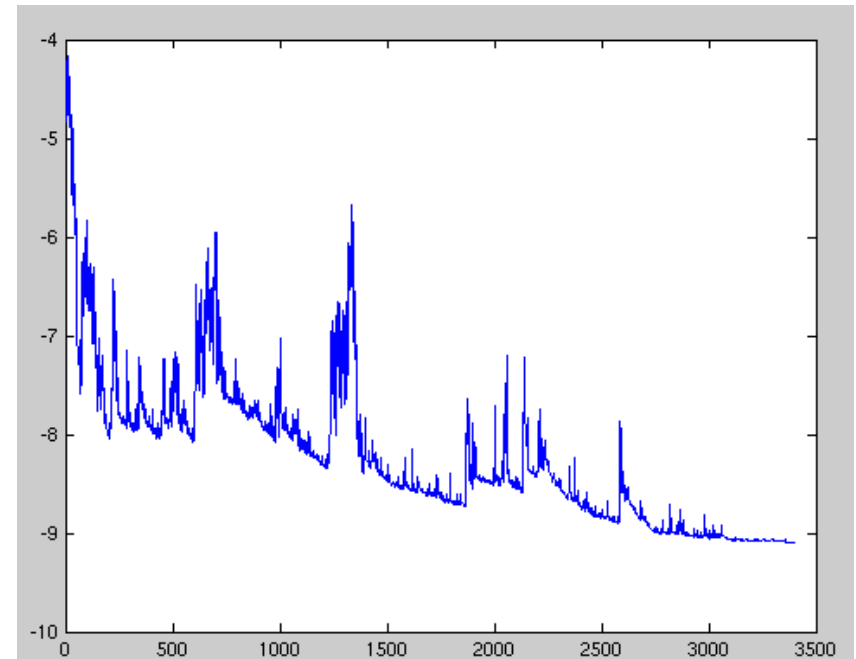
η (eta) is learning rate > 0

- Goal:
 - Minimize error
- Gradient descent
 - Calculate current error
 - Calculate derivation with respect to parameter
 - Change weights

$$w_i \leftarrow w_i - \Delta w_i = w_i - \eta \frac{\partial E}{\partial w_i}$$

- Stochastic Gradient descent
 - Random take one example
 - Update weights

$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} := \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} - \eta \begin{bmatrix} \frac{\partial}{\partial w_1} (w_1 + w_2 x_i - y_i)^2 \\ \frac{\partial}{\partial w_2} (w_1 + w_2 x_i - y_i)^2 \end{bmatrix} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} - \eta \begin{bmatrix} 2(w_1 + w_2 x_i - y_i) \\ 2x_i(w_1 + w_2 x_i - y_i) \end{bmatrix}.$$



Back-propagation

- All functions are non-linear and need to be differentiable.
- Calculate contribution of every weight to the error.
- Convergence requires derivative (changes) of error with respect to weight changes needs to go to zero.

Finding derivative of the error:

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial w_{ij}} = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial \text{net}_j} \frac{\partial \text{net}_j}{\partial w_{ij}}$$

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}} = -\eta o_i \delta_j$$

With $\eta > 0$ Always decreases $\rightarrow$ convergence

$$E = L(t, y)$$

where

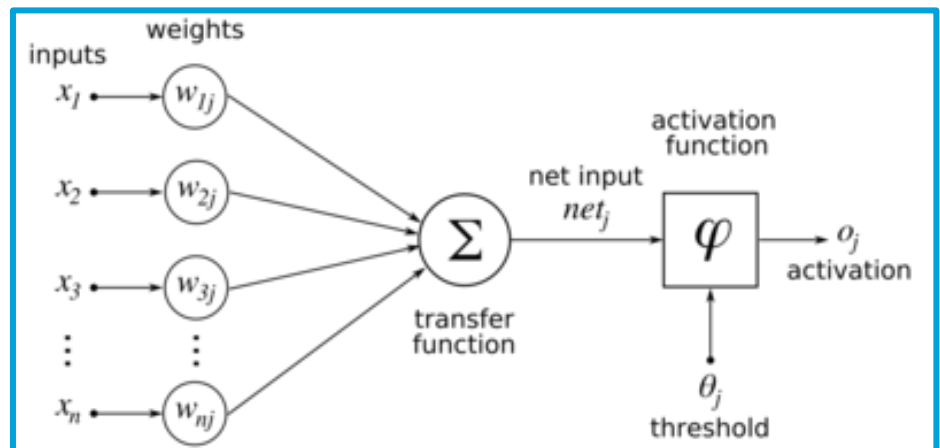
L is the loss for the output y and target value t ,
 t is the target output for a training sample, and
 y is the actual output of the output neuron.

For each neuron j , its output o_j is defined as

$$o_j = \varphi(\text{net}_j) = \varphi \left(\sum_{k=1}^n w_{kj} o_k \right),$$

Activation
function:

$$\varphi(z) = \frac{1}{1 + e^{-z}}$$



Back Propagation & Deep Learning

- All Deep Learning models are trained using the same principles as used in Back Propagation.
- There are often multiple gradients, and a few other slight modifications can be observed. But the process is more or less similar.

BERT



Maastricht University

Pre-training and Fine-Tuning BERT

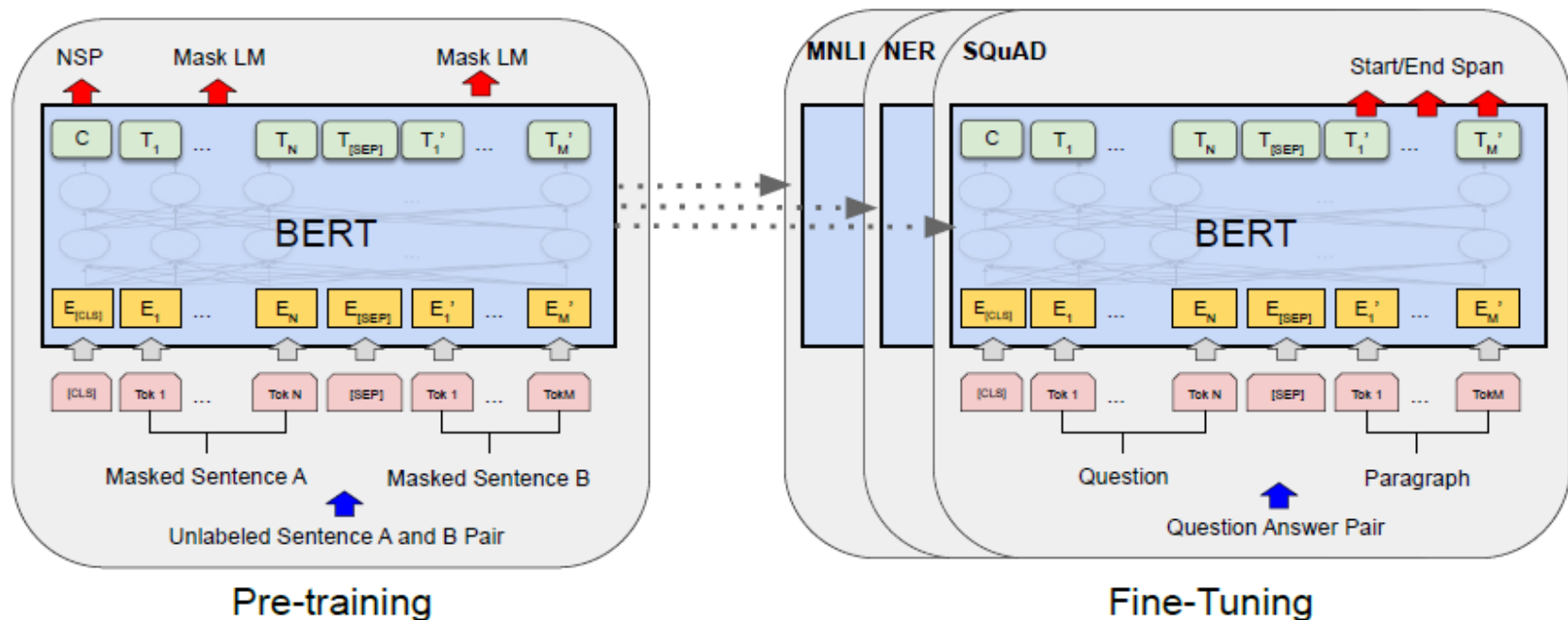


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. [CLS] is a special symbol added in front of every input example, and [SEP] is a special separator token (e.g. separating questions/answers).

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Jacob Devlin et al., May 2019.

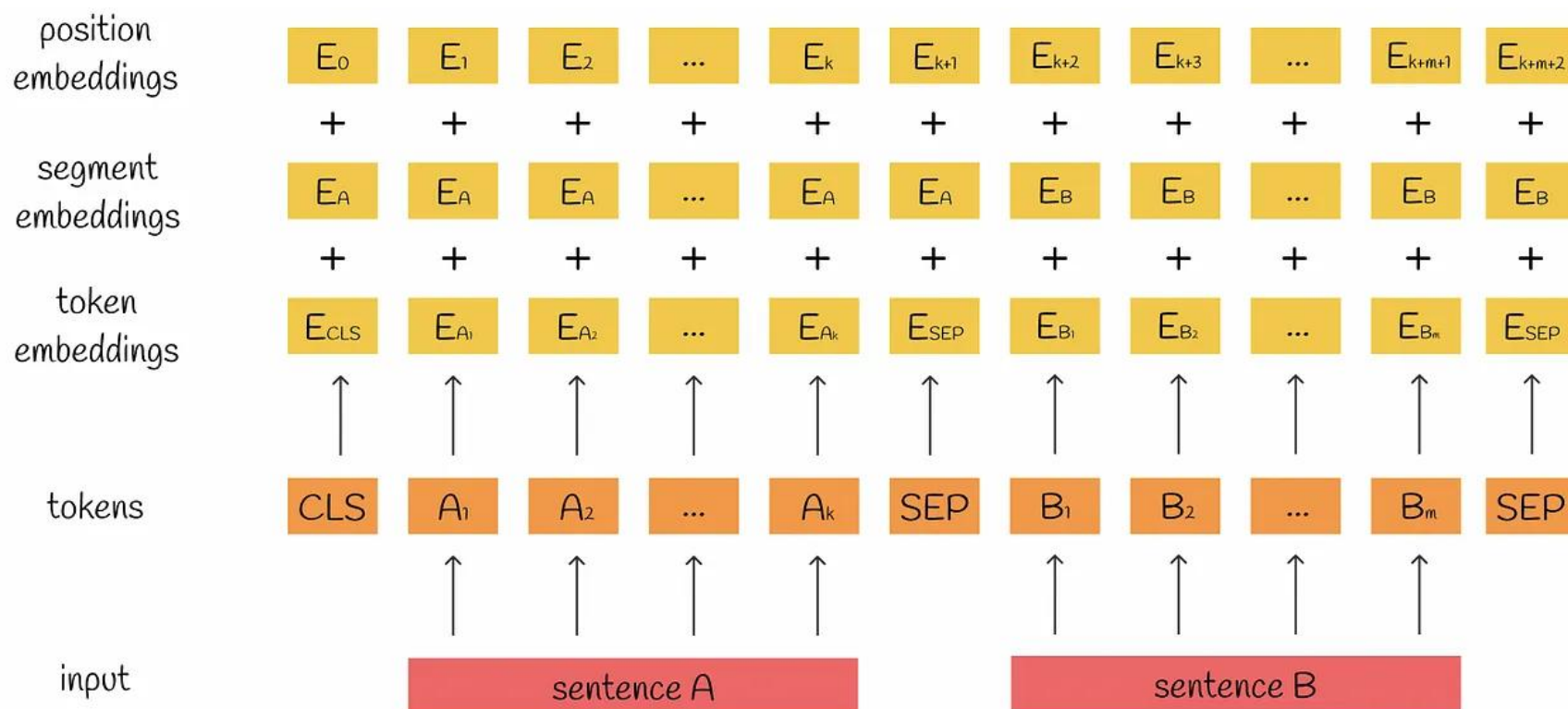
<https://arxiv.org/pdf/1810.04805.pdf>

Pre-training and fine-tuning BERT: input

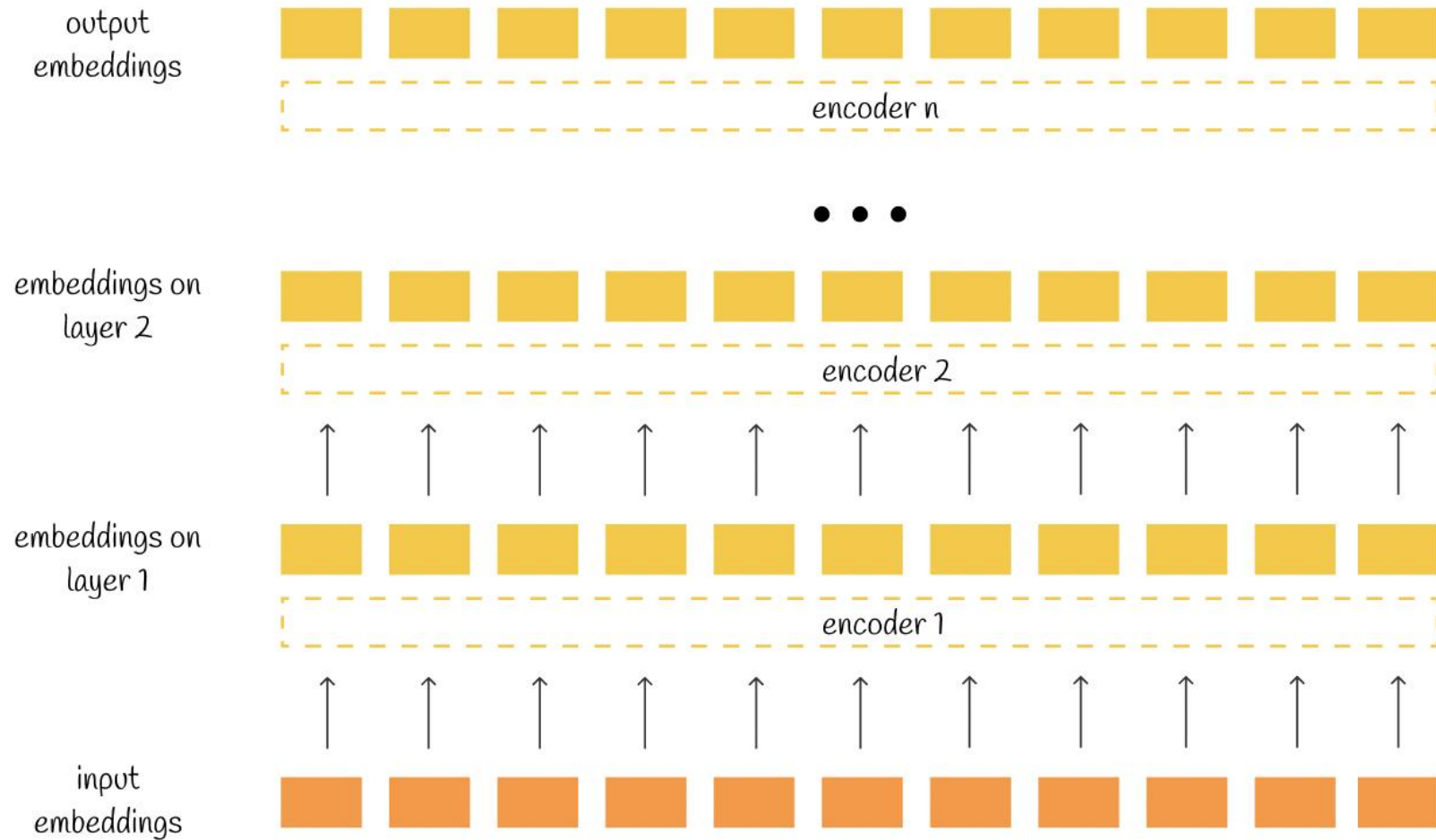
- For the input, BERT takes a single sequence or a pair of sequences. Each sequence is split into tokens.
- Additionally, two special tokens are passed to the input:
 - [CLS] — passed before the first sequence indicating its beginning: CLEAR SENTENCE. At the same time, [CLS] is also used for a classification objective during training. [SEP] — passed between sequences to indicate the end of the first sequence and the beginning of the second: SEPARATOR.
- Sending two sequences makes it possible for BERT to handle a variety of tasks where an input contains a pair of sequences (e.g. question and answer, hypothesis and premise, etc.).

Pre-training and fine-tuning BERT: Embeddings

After tokenization, an embedding is built for each token. To make input embeddings more representative, BERT constructs three types of embeddings for each token: **Token embeddings**, **segment embeddings** and **positional embeddings**.



Pre-training and Fine-Tuning BERT: Layers



Pre-Training and Fine-Tuning BERT

- **Pre-training:** BERT is pre-trained on unlabeled pair of sequences over two prediction tasks:
 - masked language modeling (MLM)
 - natural language inference (NLI).

We will discuss in detail what this is in the subsequent slides. For each pair of sequences, the model makes predictions for these two tasks and based on the loss values, it performs backpropagation to update weights.

- **Fine-tuning:** BERT is initialized with pre-trained weights during the above pre-training and can then be fine-tuned for a particular problem on labeled data during the fine-tuning process.

BERT: Training Details we will discuss...

- BERT is pre-trained on BooksCorpus (800M words) and English Wikipedia (2,500M words). For extracting longer continuous texts, authors took from Wikipedia only reading passages ignoring tables, headers and lists.
- BERT is trained on a million batches of size equal to 256 sequences which is equivalent to 40 epochs on 3.3 billion words. Each sequence contains up to 128 (90% of the time) or 512 (10% of the time) tokens.
- The training parameters are the following:
 - Optimizator: Adam (learning rate $l = 1e-4$, weight decay $L_2 = 0.01$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-6$).
 - Learning rate warmup is performed over the first 10.000 steps and then reduced linearly.
 - Dropout ($\alpha = 0.1$) layer is used on all layers.
 - Activation function: GELU.
 - Training loss is the sum of mean MLM and mean next sentence prediction likelihoods.

BERT: Masked-Language Modeling (MLM)

1. 15% of tokens are randomly masked. The masked tokens will be then predicted at the end of the iteration.

2. The masked tokens are replaced in one of three ways:

- -80% of the tokens are replaced by the **[MASK]** token.
 - Example: I bought a book → I bought a [MASK]
- -10% of the tokens are replaced by a random token.
 - Example: He is eating a fruit → He is drawing a fruit
- 10% of the tokens remain unchanged.
 - Example: A house is near me → A house is near me

3. All tokens are passed to the BERT model which outputs an embedding for each token it received as input.

4. Output embeddings corresponding to the tokens processed at step 2 are independently used to predict the masked tokens. The result of each prediction is a probability distribution across all the tokens in the vocabulary.

5. The cross-entropy loss is calculated by comparing probability distributions with the true masked tokens.

6. The model weights are updated by using backpropagation.

Masked Language Modeling

Masked language modeling is a fill-in-the-blank task, where a model uses the context words surrounding a [MASK] token to try to predict what the [MASK] word should be.

The model shown here is **BERT**, the first large transformer to be trained on this task. Enter text with one or more "[MASK]" tokens and the model will generate the most likely substitution for each.

Sentence:

It is [MASK] to
[MASK] that

Mask 1 Predictions:

11.5% **important**
11.1% **difficult**
8.8% **easy**
5.0% **possible**
3.5% **hard**

Mask 2 Predictions:

22.9% **say**
15.5% **note**
5.7% **see**
3.3% **suggest**
2.1% **conclude**

BERT: Natural Language Inference (NLI)

- BERT learns how to predict the second sequence follows the first. The prediction is made by using only the embedding from the final hidden state of the [CLS] token which is supposed to contain aggregated information from both sequences: **Question: How is this represented?**
- Similarly to Mask Language Modeling, a constructed probability distribution (binary in this case) is used to calculate the model's loss and update the weights of the model through backpropagation.
- For NLI, in the original BERT paper, it is recommended choosing 50% of pairs of sequences which follow each other in the corpus (positive pairs) and 50% of pairs where sequences are taken randomly from the corpus (negative pairs).



We will get back to this strategy later in the section on
“contrastive learning”

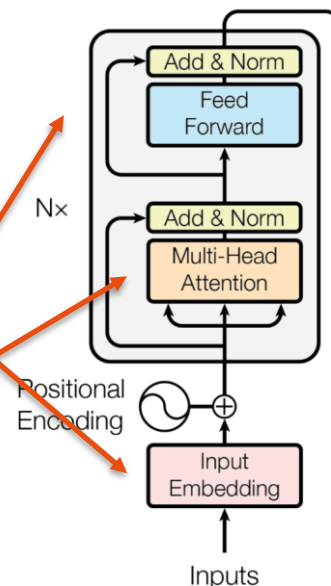
Which Weights are Updated during Training in BERT?

Backpropagation is the process of training Artificial Neural Networks (ANNs) by adjusting the weights of each layer in order to minimize a predefined error or cost function. Steps involved in backpropagation for pre-training and fine-tuning BERT are:

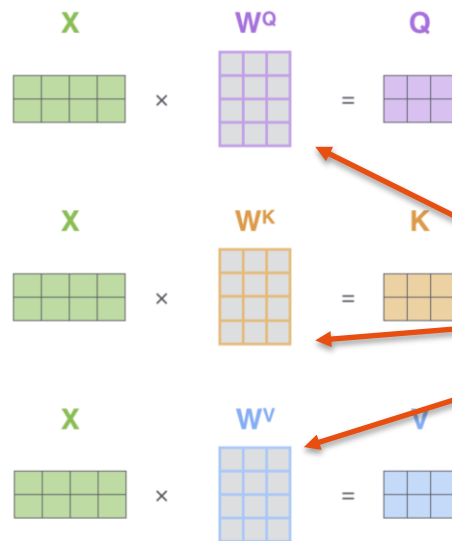
1. Feed forward: The input data is sent forward through the network, each layer performing some operation on the data. At the end of this process, an output is generated.
2. Calculate the error: The generated output is compared to the expected output (also known as the target) and an error is calculated.
3. Calculate the derivatives: The derivatives of the error with respect to the weights of each layer of the network is calculated.
4. Backpropagate the error: The derivatives are used to propagate the error backwards through the network. This helps in calculating the adjustments that need to be made to the weights in order to minimize the error.
5. Update the weights: The weights of each layer of the network are updated in the direction that helps to minimize the error.

The number of weights per layer depends on the model architecture. For example, BERT Base has 12 layers, each with its own set of weights:

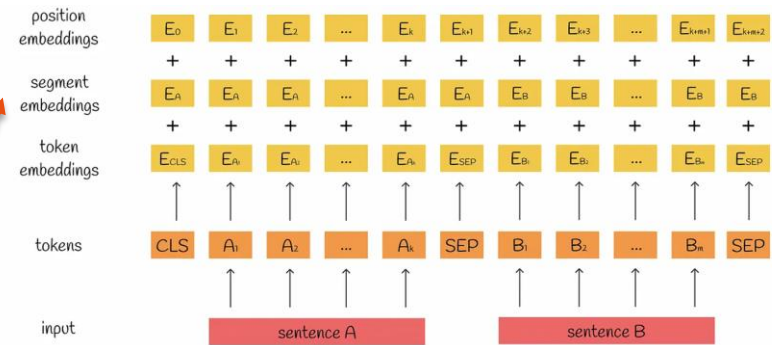
- The weights for the embeddings layer are also updated since this layer is a part of the network and is used to represent the input data.
- The weights for the Q, K, and V matrices are also updated as part of the backpropagation process.
- The feed forward neural network layer also has its own set of weights which are updated as part of the backpropagation process.



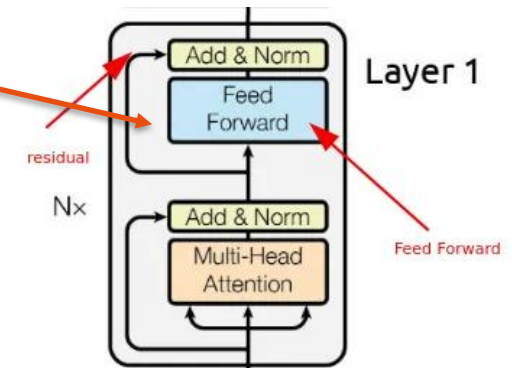
Weights that are updated in BERT



- The weights for the **embeddings layer** are also updated since this layer is a part of the network and is used to represent the input data.
- The weights for the **W^Q , W^K , and W^V matrices** are also updated as part of the backpropagation process for the proper self-attention calculations.
- The **feed forward neural network** layer also has its own set of weights which are updated as part of the backpropagation process.



How many trainable weights do you estimate that BERT base has: 12 layers, 768 dim embeddings and 12 attention heads.

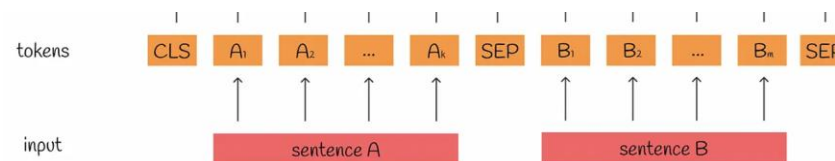


How many trainable Weights in BERT-base?

Let's start with the “Embedding Layer”

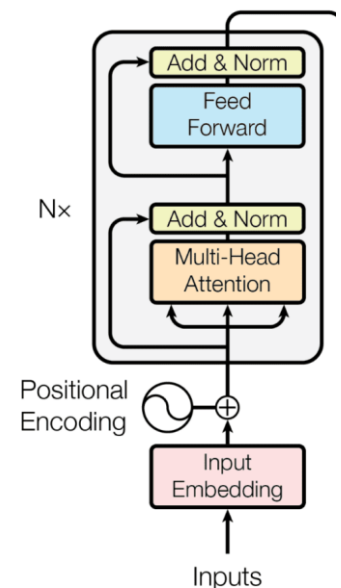
BERT uses a word embedding layer (vocabulary size: V) and positional embeddings.

- **Token Embeddings:** $V \times 768$
- **Positional Embeddings:** $P \times 768$ (usually $P=512$, the maximum sequence length)
- **Segment Embeddings:** 2 segments (A/B) $\times 768$



Embedding layer total:

$$\text{Embeddings} = V \times 768 + P \times 768 + 2 \times 768$$

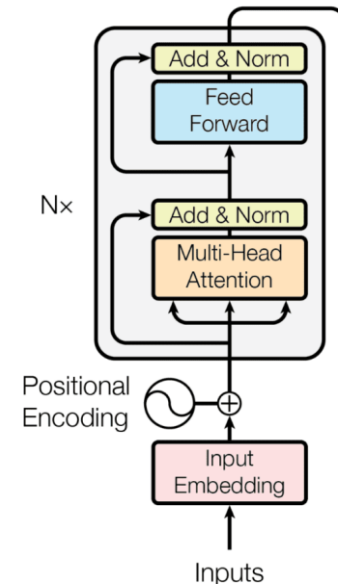


A closer look at the Embeddings during training

During training:

- **Token Embeddings:** These are updated. The model learns better token representations by optimizing these embeddings during pretraining tasks (Masked Language Modeling and Next Sentence Prediction). For example:
 - If "apple" is masked and predicted as "fruit," the corresponding embedding for "apple" is updated to better encode its contextual meaning.
- **Positional Embeddings:** These are also updated. While they encode fixed positional information initially, their representations can be fine-tuned to optimize performance on the training data.
- **Segment Embeddings:** These are updated as well. For tasks like Next Sentence Prediction or any task that uses segments, the model learns to adjust these embeddings to capture relationships between sentence pairs.

In summary, all components of the embedding layer are trainable and are updated during training.



. Why Not Just Update Segment Embeddings?

If only the segment embeddings were trainable:

- The model would fail to adapt the token embeddings to learn nuanced word meanings and their contexts.
- Positional embeddings would remain static, potentially limiting the model's ability to encode positional relationships effectively for different tasks.
- The model's performance on downstream tasks would be significantly worse.

Trainable Weights in BERT-base: 2. Transformer Layers 1/2:

There are 12 transformer layers, each with:

- **Self-Attention Weights:**

- Query, Key, Value projections (768×768 each) for each attention head:

$$Q/K/V = 3 \times 768 \times 768$$

- Output projection back to hidden size (768×768):

$$\text{Output Projection} = 768 \times 768$$

Total for self-attention:

$$\text{Self-Attention Layer} = (3 \times 768^2) + 768^2 = 4 \times 768^2$$

Trainable Weights in BERT-base: 2. Transformer Layers 2/2

- **Feedforward Layers:** Two fully connected layers in the feedforward network:

- Input to intermediate (768×3072):

$$\text{Intermediate FC} = 768 \times 3072$$

- Intermediate back to hidden size (3072×768):

$$\text{Output FC} = 3072 \times 768$$

Total for feedforward:

$$\text{Feedforward} = 768 \times 3072 + 3072 \times 768 = 2 \times 768 \times 3072$$

- **Layer Norms:** Two LayerNorm parameters per layer:

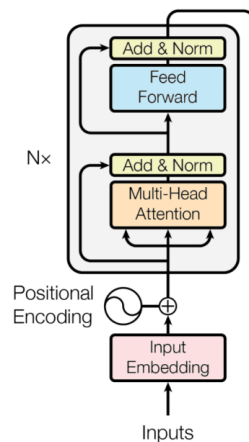
$$\text{LayerNorms} = 2 \times 768 \times 2 = 2 \times 768$$

Total per transformer layer:

$$\text{Per Layer Total} = (4 \times 768^2) + (2 \times 768 \times 3072) + (2 \times 768)$$

The **3072** comes from the fact that in BERT, the hidden size of 768 is expanded by a factor of **4** in the intermediate layer of the feedforward network. This design choice is based on the original Transformer architecture in the Vaswani et al. (2017) paper.

Two fully connected layers



Trainable Weights in BERT-base: Final Calculation:

Combine all layers and embeddings:

$$\text{Total Parameters} = \text{Embeddings} + 12 \times \text{Per Layer Total}$$

Assuming typical BERT-base settings:

- Vocabulary size (V) = 30,000
- Positional embeddings (P) = 512

Embeddings:

$$\text{Embeddings} = (30,000 \times 768) + (512 \times 768) + (2 \times 768) = 23,187,456$$

Per Layer:

$$\begin{aligned} \text{Per Layer Total} &= (4 \times 768^2) + (2 \times 768 \times 3072) + (2 \times 768) \\ &= (4 \times 589,824) + (2 \times 2,359,296) + 1,536 = 2,359,296 + 4,718,592 + 1,536 = 7,080,192 \end{aligned}$$

Total Parameters:

$$\text{Total Parameters} = 23,187,456 + (12 \times 7,080,192) = 23,187,456 + 84,962,304 = 108,109,760$$

~ 100.000.000 trainable weights!

Typical Pre-Training:

1. Dataset Size

The original BERT-base model was pretrained on a corpus containing **3.3 billion words**, combining:

- BooksCorpus (~800M words)
- English Wikipedia (~2.5B words)

2. Number of Training Steps

The original BERT-base pretraining was done for **1,000,000 steps** with:

- A batch size of **256 sequences**
- Sequence length **512 tokens**



See why 2 slides
further...

This corresponds to processing about **40 epochs** over the dataset, assuming 3.3 billion words. The exact formula to estimate epochs is:

$$\text{Epochs} = \frac{\text{Steps} \times \text{Batch Size} \times \text{Sequence Length}}{\text{Total Tokens in Corpus}}$$

3. Recommended Epochs

While 40 epochs worked for the original BERT training, it might not always be necessary for other datasets or training setups. Some recommendations include:

- **Smaller Corpora:** Aim for 10-20 epochs for smaller datasets.
- **Large-scale Pretraining:** Expect 40-50 epochs for very large and diverse datasets.
- **Domain-specific Pretraining:** Focus on fewer epochs (e.g., 5-10) if you're fine-tuning BERT on domain-specific data like legal or medical text.

4. Compute Requirements

Pretraining for the full 1M steps took:

- **16 TPU v3 chips** (for ~4 days)
- Equivalent to several weeks on high-end GPUs like NVIDIA A100s.



For domain-specific adaptations (e.g., LegalBERT or BioBERT), you can often stop after **5-10 epochs** if you're starting from an existing pretrained checkpoint.

Why 1.000.000 training steps?

1. Dataset Size

BERT was pretrained on:

- **BooksCorpus:** ~800 million words.
- **English Wikipedia:** ~2.5 billion words.
- **Total words:** ~3.3 billion.

For tokenization, BERT uses WordPiece, splitting words into subword units. On average, one word corresponds to **1.1 to 1.2 tokens**. Therefore, the total number of tokens in the corpus is:

$$\text{Total Tokens} \approx 3.3 \text{ billion words} \times 1.1 = 3.63 \text{ billion tokens.}$$

2. Batch Size and Sequence Length

Each training step processes:

- **Batch size:** 256 sequences.
- **Sequence length:** 512 tokens.

This means each training step processes:

$$\text{Tokens per step} = 256 \times 512 = 131,072 \text{ tokens.}$$

3. Total Training Steps

To determine the number of steps, we calculate how many tokens need to be processed to cover the entire dataset for a single epoch:

$$\text{Tokens per epoch} = 3.63 \text{ billion tokens.}$$

With a batch size of 256 sequences and sequence length of 512, the number of steps per epoch is:

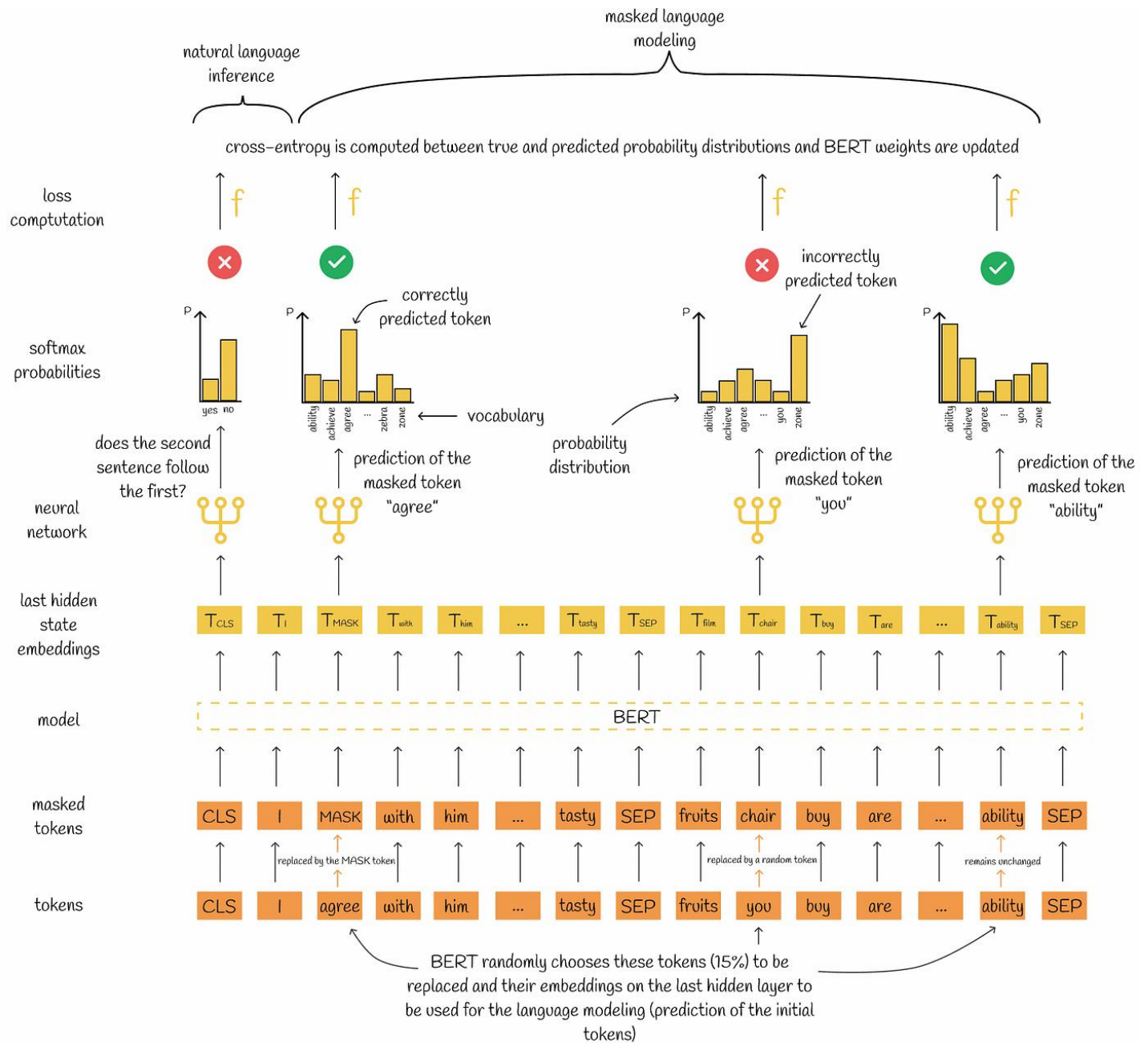
$$\text{Steps per epoch} = \frac{\text{Tokens per epoch}}{\text{Tokens per step}} = \frac{3.63 \times 10^9}{131,072} \approx 27,686.$$

The authors trained BERT for approximately **40 epochs** over the dataset, which gives:

$$\text{Total Steps} = 40 \times 27,686 \approx 1,107,440.$$

This is rounded to **1,000,000 steps**, which aligns with the computational budget and convergence observed in practice.

BERT: Pre-training for Natural Language Inference with Masked Language Modeling



BERT: After Pre-Training ...

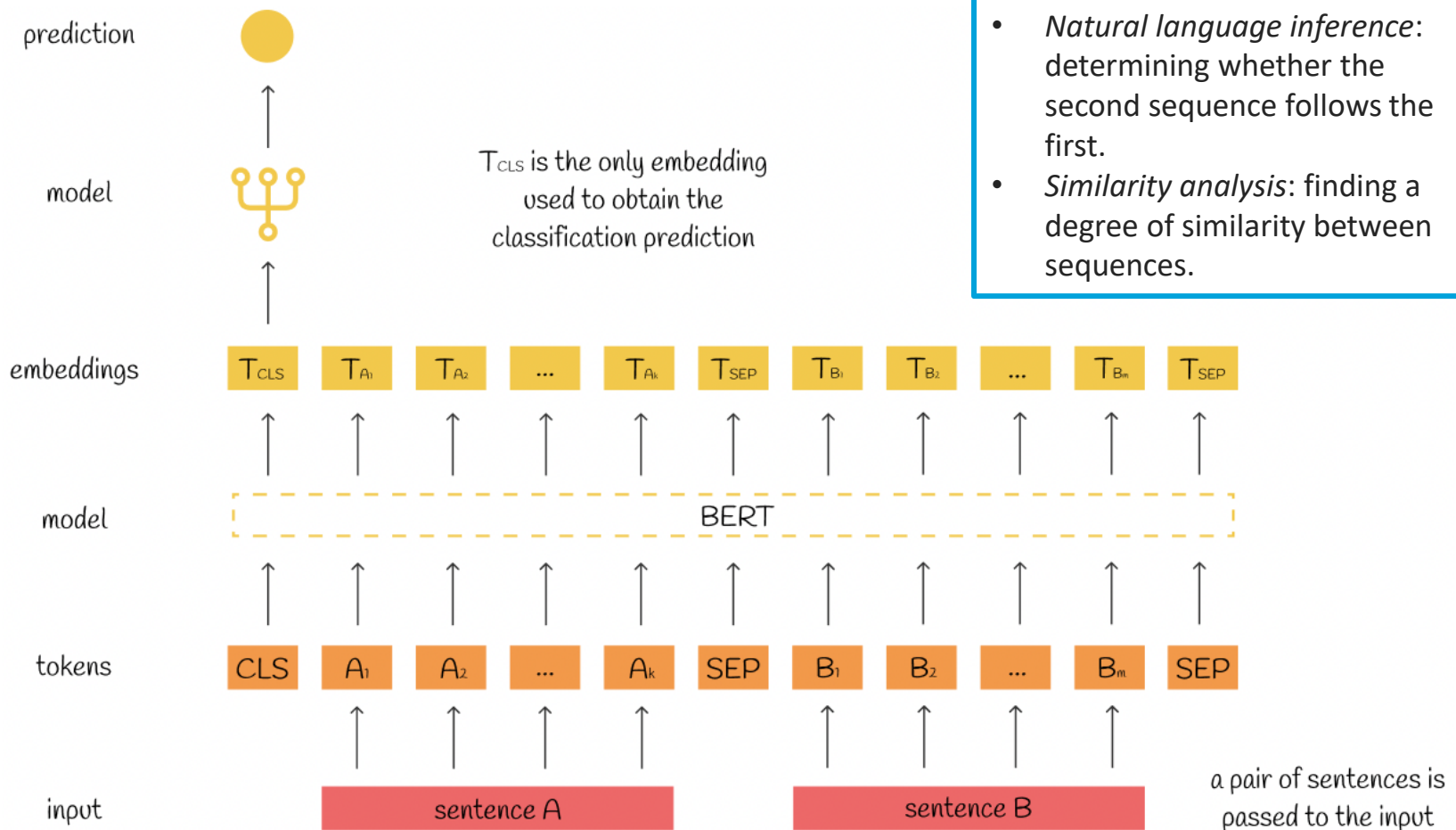
- Once pre-training is completed, BERT can understand semantic meanings of words and construct embeddings which can almost fully represent their meanings.
- Fine-tuning will gradually modify BERT weights for solving a particular downstream task.
- BERT's ability to build bidirectional text representations is a major advantage dealing with linguistic phenomena such as homonyms and other ambiguous long-distance references (pronouns, co-references, ...).

Now, let's take a look at a number of fine-tuning operations for:

1. Sentence Pair Classification
2. Question-Answer Tasks
3. Single Sentence Classification
4. Single Sentence Tagging



BERT: Sentence Pair Classification

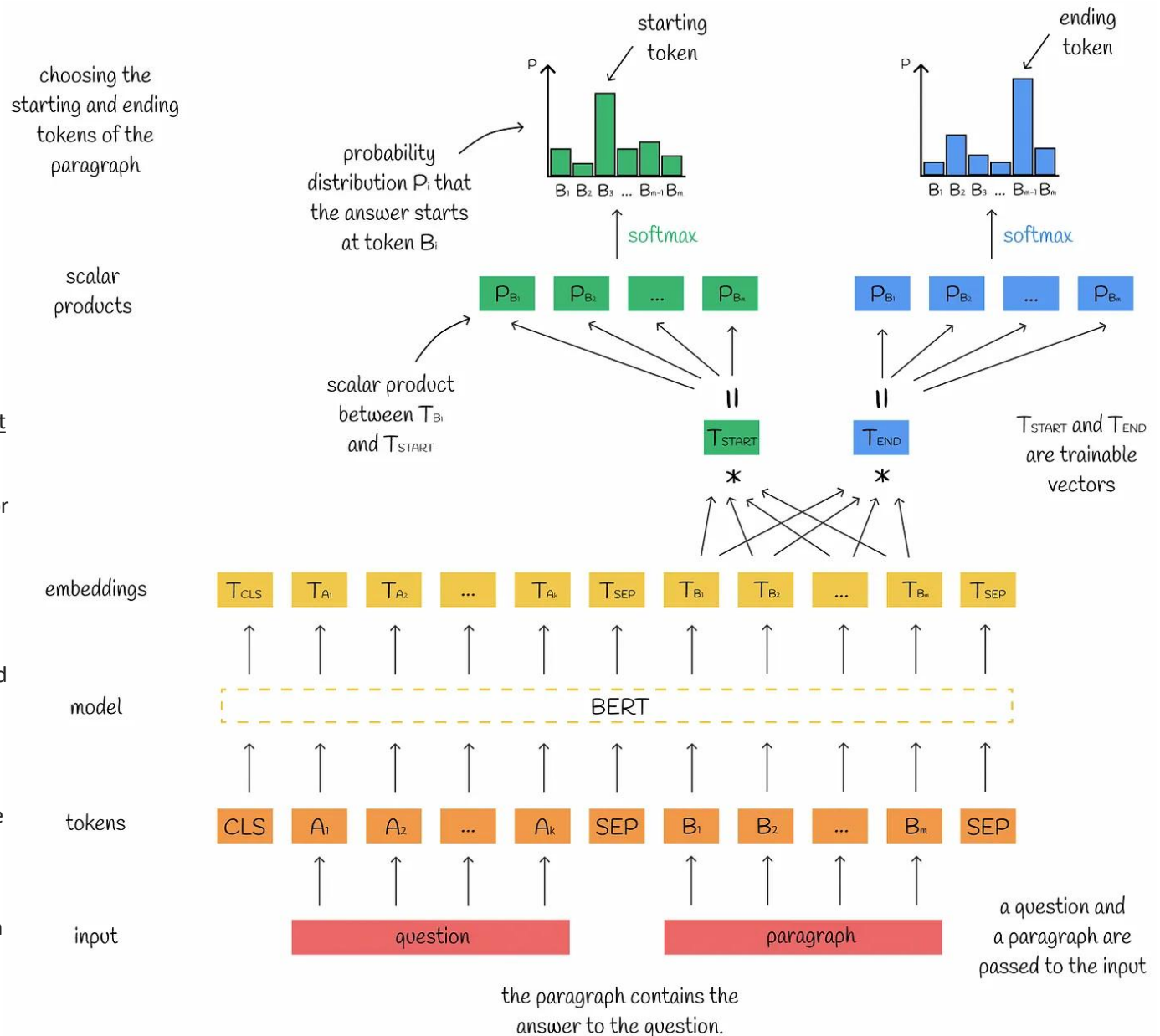


BERT:

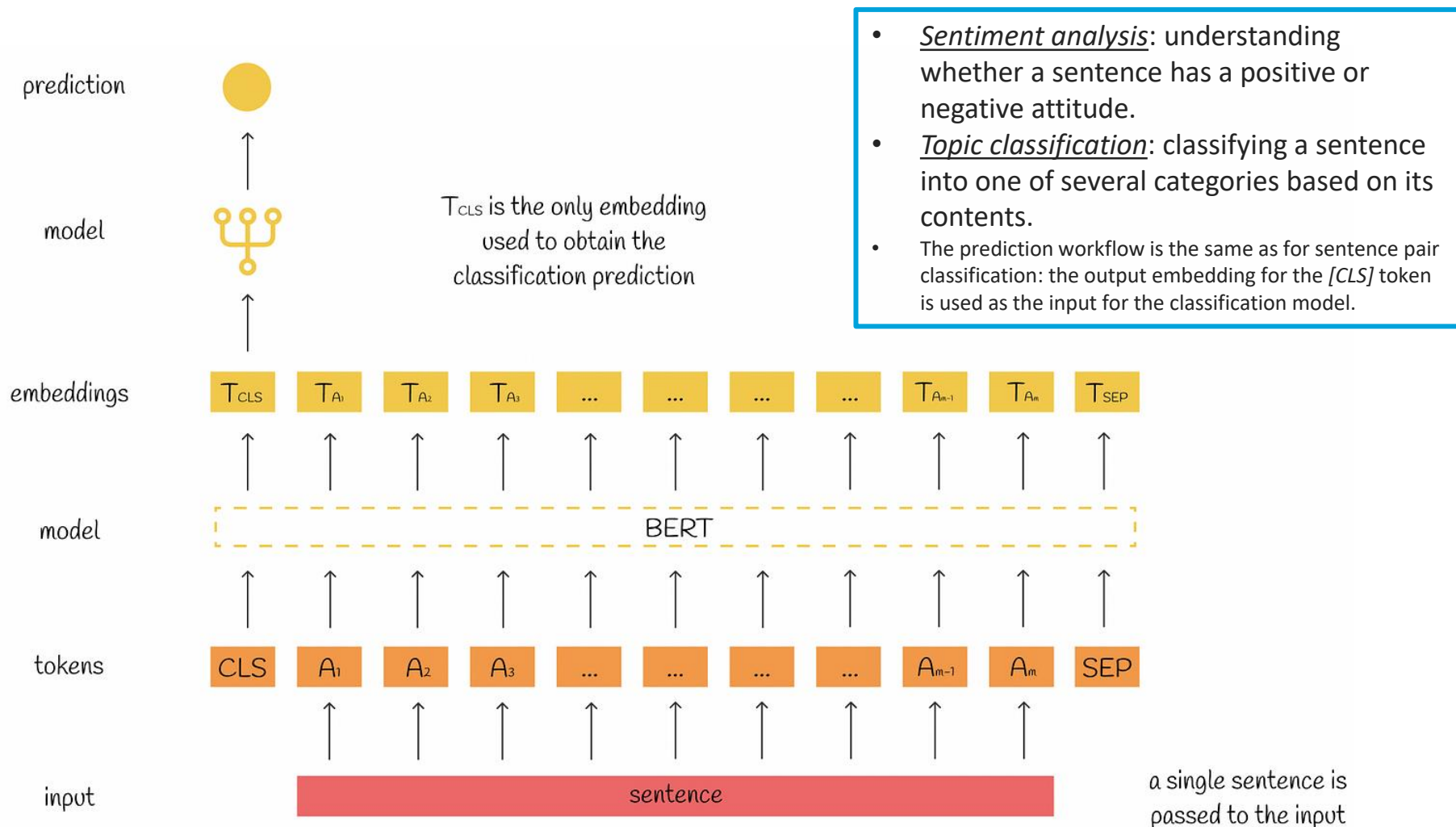
Question Answering Task

Find an answer in a text paragraph corresponding to a particular question.

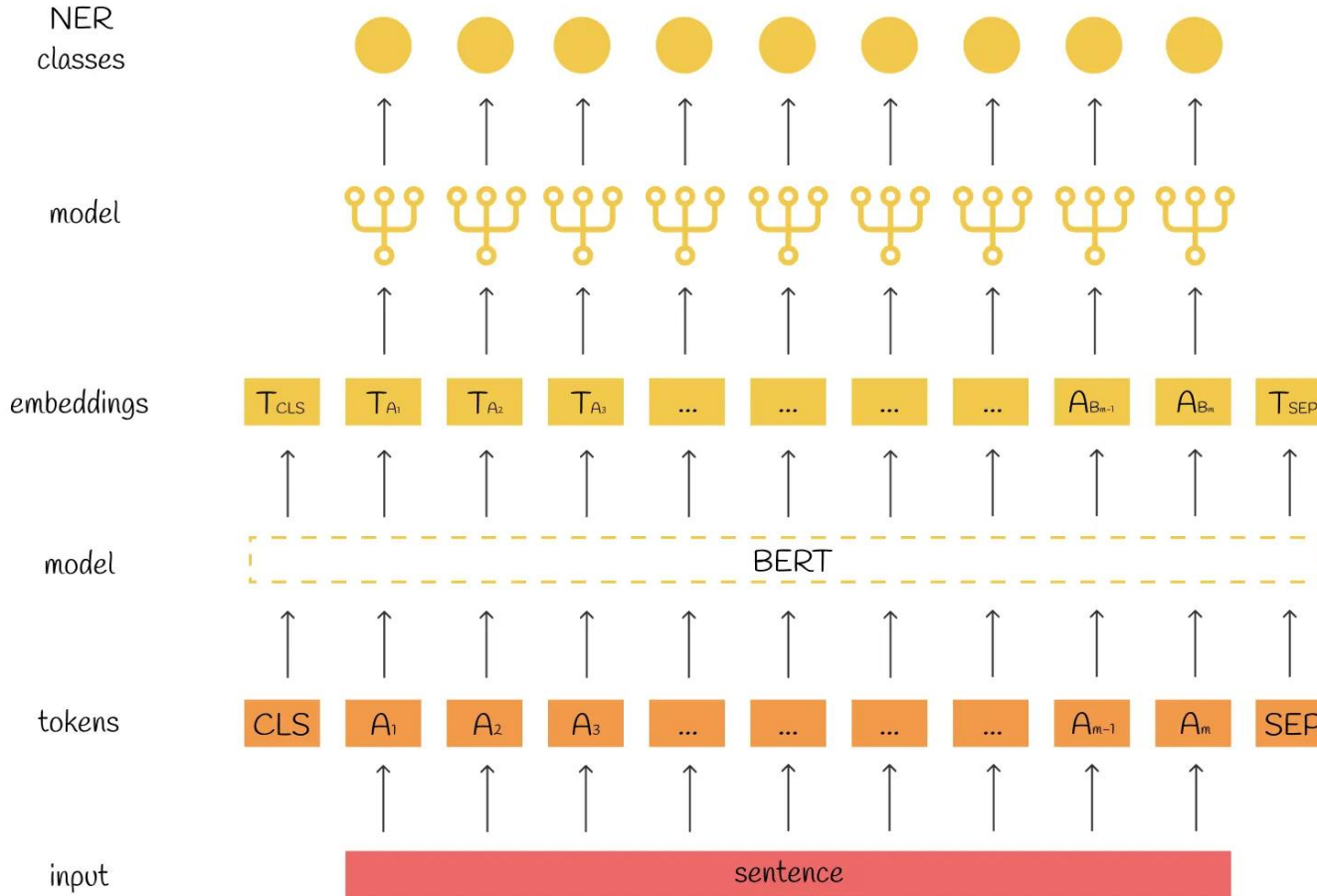
For finding a position of the start answer token in the paragraph, the scalar product between every output embedding and a special trainable vector T_{start} is calculated. For most cases when the model and the vector T_{start} are trained accordingly, the scalar product should be proportional to the likelihood that a corresponding token is in reality the start answer token. To normalize scalar products, they are then passed to the SoftMax function and can be thought as probabilities. The token embedding corresponding to the highest probability is predicted as the start answer token. Based on the true probability distribution, the loss value is calculated, and the backpropagation is performed. The analogous process is performed with the vector T_{end} for predicting the end token.



BERT: Single Sentence Classification



BERT: Single Sentence Tagging



Named entity recognition (NER) is a machine learning problem example, as is Part of Speech (POS) tagging.

Embeddings are computed for tokens of an input sentence, as usual. Then every embedding (except for $[CLS]$ and $[SEP]$) is passed independently to a model which maps each of them to a given NER class or POS tag (or not, if it cannot). Next, map every token of a sequence to one of respective entities or POS tags

a single sentence is passed to the input

Various BERT models

Model	Parameters (Millions)
BERT-Tiny	4.4
BERT-Mini	11.3
BERT-Small	29.1
BERT-Medium	41.7
BERT-Base	110.1
BERT-Large	340



Comparison	BERT October 11, 2018	RoBERTa July 26, 2019	DistilBERT October 2, 2019	ALBERT September 26, 2019
Parameters	Base: 110M Large: 340M	Base: 125 Large: 355	Base: 66	Base: 12M Large: 18M
Layers / Hidden Dimensions / Self-Attention Heads	Base: 12 / 768 / 12 Large: 24 / 1024 / 16	Base: 12 / 768 / 12 Large: 24 / 1024 / 16	Base: 6 / 768 / 12	Base: 12 / 768 / 12 Large: 24 / 1024 / 16
Training Time	Base: 8 x V100 x 12d Large: 280 x V100 x 1d	1024 x V100 x 1 day (4-5x more than BERT)	Base: 8 x V100 x 3.5d (4 times less than BERT)	[not given] Large: 1.7x faster
Performance	Outperforming SOTA in Oct 2018	88.5 on GLUE	97% of BERT-base's performance on GLUE	89.4 on GLUE
Pre-Training Data	BooksCorpus + English Wikipedia = 16 GB	BERT + CCNews + OpenWebText + Stories = 160 GB	BooksCorpus + English Wikipedia = 16 GB	BooksCorpus + English Wikipedia = 16 GB
Method	Bidirectional Transformer, MLM & NSP	BERT without NSP, Using Dynamic Masking	BERT Distillation	BERT with reduced parameters & SOP (not NSP)



Why Domain Dependent BERT Embeddings?

Bible:

The concept, “act of God,” is central to the biblical understanding of God and his relation to the world. Repeatedly we are told of the great works performed by God in behalf of his people and in execution of his own purposes in history.

“... an Act of God”

Legal Agreement:

e. Cancellations: Lyonpride is not liable for return of RMHB fees in the event of cancellations or other defaults by Lyonpride due to Acts of God. The RMHB does not have the right to terminate this Agreement for any delay or default in performing hereunder if such delay or default is caused by conditions beyond the control of Lyonpride including, but not limited to Acts of God.

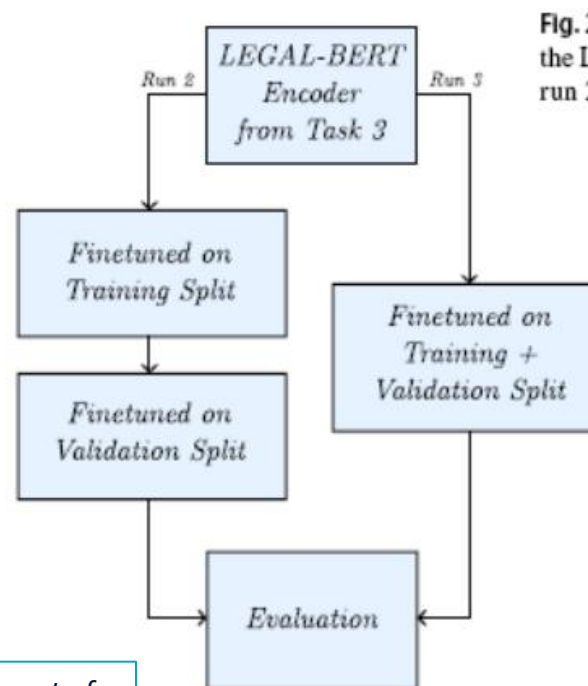


Fig. 2 Fine-tuning workflow of the LEGAL-BERT encoder for run 2 and 3

What to do with Domain Dependent Embeddings?

1. Use the embeddings in the fine-tuned BERT LLM model.
2. Extract the embeddings and use in T-5 (better translations), Pegasus (better summaries), GPT (better text generation), or other transformer-based model.

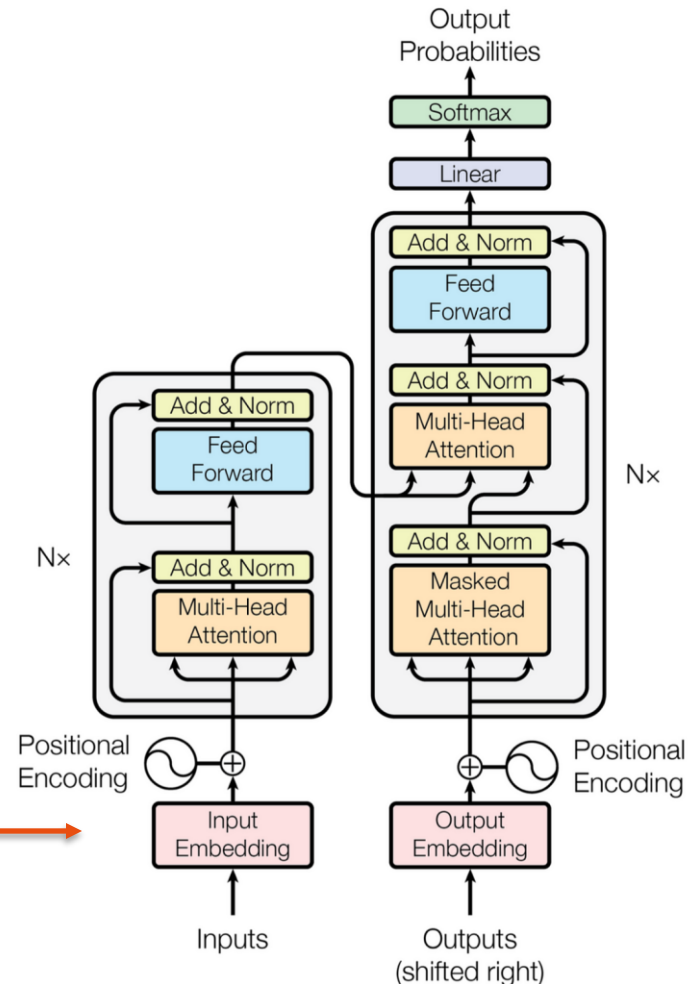
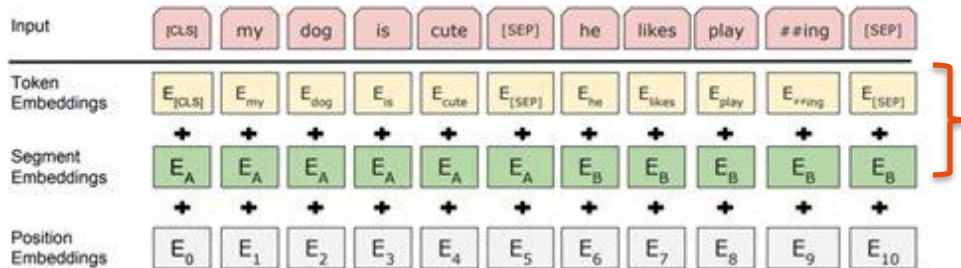


Figure 1: The Transformer - model architecture.

GPT-2

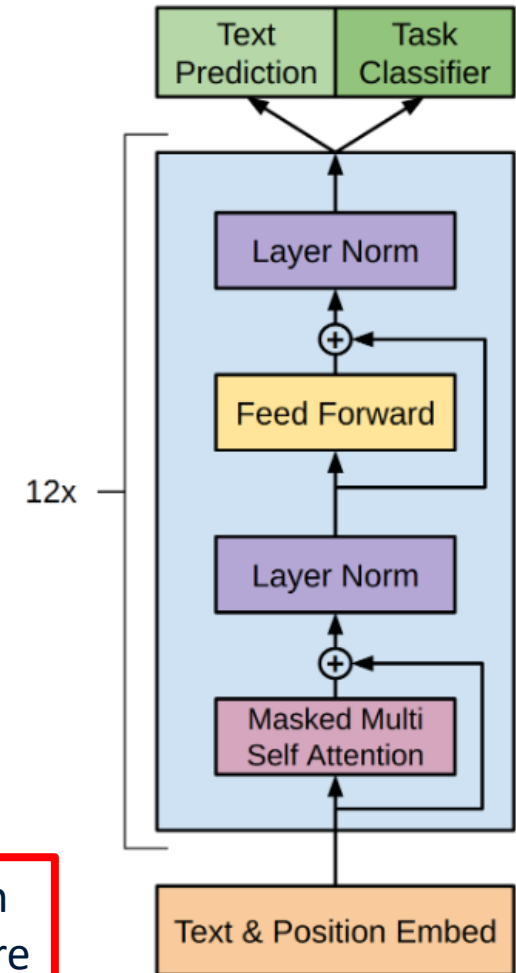


Maastricht University

Decoder only Model (GPT-2/3/4)

- The GPT model only uses decoder part.
- GPT is trained with decoder-only architecture and a standard left-to-right language modelling objective on a large text corpus where the objective is to predict the next token, considering the previous tokens.
- Self-attention is done on the prompt (or a history of the conversation in case of ChatGPT).
- GPT is autoregressive inference model, so it need to iteratively generate one token at a time.
- These models can achieve state-of-the-art few-shot performance across various linguistic tasks.
- Decoder only best for TEXT-GENERATION based on few/zero shot learning (e.g.: the “prompt”).

Self-Attention Encoder is the text in the prompt and in ChatGPT the entire conversation (up to max length context)



Pre-Train GPT-2

- You can only run GPT-2 on your local computer, cloud, or google Colab.
- You cannot train GPT-3, ChatGPT, or GPT-4 as the models are proprietary.
- The easiest way to run GPT-2 is by using Hugging Face transformers

It is mainly based on PyTorch but also supports TensorFlow and FLAX (JAX) models. Before we start, you need a functional Python 3.x environment (either vanilla Python or Anaconda, we use the former).

Follow the installation instructions here. Namely, for the vanilla Python with PIP, type in your terminal:

```
pip3 install torch 'transformers[torch]'
```

For Anaconda, type:

```
conda install -c huggingface transformers
```

Preparing the Training Data for GPT

- GPT models are trained in an unsupervised way on a large amount of text (or text corpus).
- The corpus is broken into sequences, usually of uniform size (e.g., 1024 tokens each).
- The model is trained to predict the next token (word) at each step of the sequence.

For example (here, we write words instead of integer encodings for clarity):

position	1	2	3	4	5	6	7	8	9
input_ids	The	elf	queen	was	wearing	a	cloak	.	[END]
labels	elf	queen	was	wearing	a	cloak	.	[END]	[-1]

Pre-training

- There are many parameters to be set.
- See Huggingface documentation for more details:
https://huggingface.co/docs/transformers/main_classes/trainer.
- load the corpus for training, tokenize it and break it into 511-token pieces plus the [END] token, which brings the sequence length to 512. Next, we split the data into train and validation sets in `train_val_split()` and in `prepare_dsets()` wrap them in the PyTorch datasets.
- Training is similar to BERT
 - Predict next word
 - Back-propagate errors
 - Repeat for several epochs

Difference pre-training and fine-tuning: pre-training (or training from scratch) means starting from a randomly-initialized model with no pre-trained weights. Fine-tuning starts with weights from a pre-trained model.

```
training_args = transformers.TrainingArguments(  
    output_dir="idiot_save/",  
    learning_rate=1e-3,  
    per_device_train_batch_size=1,  
    per_device_eval_batch_size=1,  
    num_train_epochs=20,  
    evaluation_strategy='epoch',  
    save_strategy='no',  
    )
```



Fine Tuning GPT-2

- New language model trained exclusively on biomedical abstracts and papers.
- This GPT-style model can achieve strong results on a variety of biomedical NLP tasks, including a new state of the art performance of 50.3% accuracy on the MedQA biomedical question answering task.
- PubMedGPT 2.7B is an autoregressive language model with 2.7B parameters. It uses the standard GPT-2 architecture

Healthcare

Stanford CRFM Introduces PubMedGPT 2.7B



The new 2.7B parameter language model trained on biomedical literature delivers an improved state of the art for medical question answering.

Parameter	Value
hidden size	2560
heads	20
layers	32
vocab size	28896
sequence length	1024

Other examples of pre-training GPT-2 (from Scratch)

- GPT-2 from Scratch: ‘Les Misérables’ Written by GPT to the Surprise of All! : https://medium.com/@xiaohan_63326/gpt-2-from-scratch-les-mis%C3%A9rables-written-by-gpt-to-the-surprise-of-all-6c20e355a6ca
- Other examples using “minGPT” or “nanoGPT”, small, educational implementations of GPT in pyTorch:
 - <https://blogs.rstudio.com/ai/posts/2023-06-20-gpt2-torch/>
 - <https://jaketae.github.io/study/gpt/>
 - <https://www.it-jim.com/blog/training-and-fine-tuning-gpt-2-and-gpt-3-models-using-hugging-face-transformers-and-openai-api/>



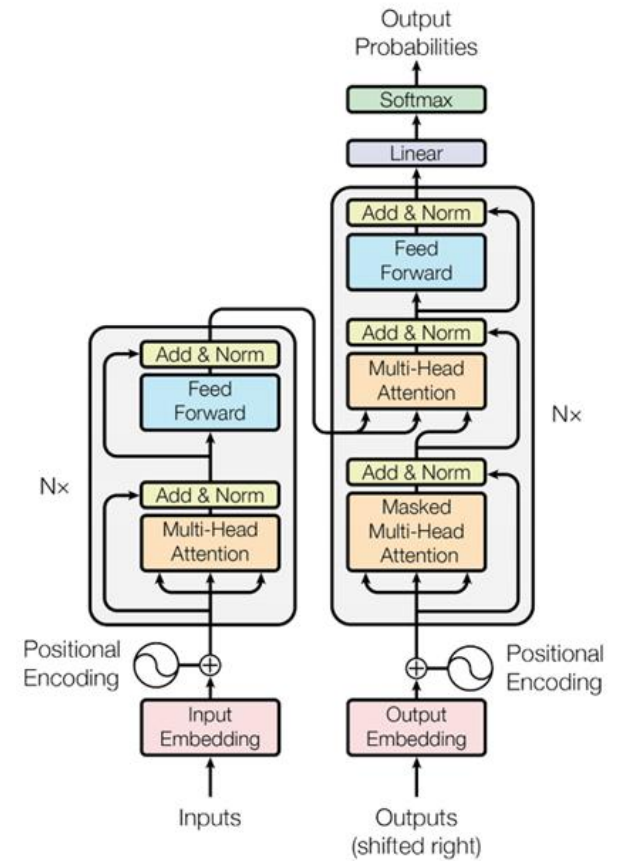
T-5



Maastricht University

Pre-training T5

- Full encoder-decoder model.
- T5 prepends a different prefix to the input corresponding to each task, e.g., for translation: translate English to German: ..., for summarization: *summarize:*
- For training, we always need an input sequence and a corresponding target sequence.
- Pretraining includes both supervised and self-supervised training:
 - Supervised training is conducted on downstream tasks provided by the GLUE and SuperGLUE benchmarks (converting them into text-to-text tasks as explained above).
 - Self-supervised training uses corrupted tokens, by randomly removing 15% of the tokens and replacing them with individual sentinel tokens (if several consecutive tokens are marked for removal, the whole group is replaced with a single sentinel token. In this setup, spans of the input sequence are masked by so-called sentinel tokens (a.k.a unique mask tokens) and the output sequence is formed as a concatenation of the same sentinel tokens and the real masked tokens. Each sentinel token represents a unique mask token for this sentence and should start with <extra_id_0>, <extra_id_1>, ... up to <extra_id_99>. As a default, 100 sentinel tokens are available in T5Tokenizer.



Original text

Thank you for inviting me to your party last week.

Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>

Which weights are Updated in T5 Back-propagation?

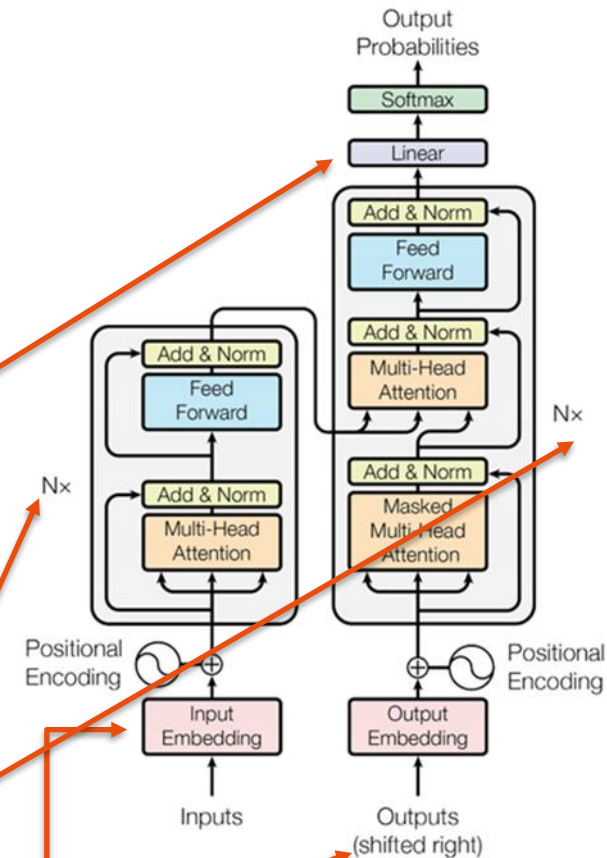
In T5, backpropagation extends to the full model, through all decoder and encoder layers up to the embedding tables.

Back-propagations:

1. Feed forward: The input data is sent forward through the network (encoder and decoder), each layer performing some operation on the data. At the end of this process, an output is generated.
2. Calculate the error: The generated output is compared to the expected output (also known as the target) and an error is calculated.
3. Calculate the derivatives: The derivatives of the error with respect to the weights of each layer of the network is calculated.
4. Backpropagate the error: The derivatives are used to propagate the error backwards through the network. This helps in calculating the adjustments that need to be made to the weights in order to minimize the error.
5. Update the weights: The weights of each layer of the network are updated in the direction that helps to minimize the error. Both for encoder and decoder.

Weights that are adjusted:

1. Final linear layer weights
2. Decoder(Feed forward layer weights & Q, K, V weight matrices 2x)
3. Encoder(Feed forward layer weights & Q, K, V weight matrices)
4. Embeddings layer



Fine-Tuning T5: Title Generation based on Text

Model training

Let's jump now into code with Colab! You can find all the code in this public [Colab notebook](#) as well. This is what we'll do:

1. Connect Google Drive to Colab to have persistent storage across Colab sessions.
2. Download the Medium dataset from Kaggle.
3. Load the dataset.
4. Split the dataset into train, validation, and test set.
5. Preprocess the dataset for T5.
6. Preparing the Hugging Face trainer.
7. Start TensorBoard.
8. Fine-tune T5.
9. Try the model.
10. Evaluate the model on the test set.

Decoder

- `title [string]`: The title of the article.
- `text [string]`: The text content of the article.
- `url [string]`: The URL associated with the article.
- `authors [list of strings]`: The article authors.
- `timestamp [string]`: The publication datetime of the article.

Encoder

Step	Training Loss	Validation Loss	Rouge1	Rouge2	RougeL	Perplexity	Gen. Len.
100	3.078800	2.602307	28.613000	13.896000	25.131000	28.240700	12.302000
200	2.743800	2.520881	30.788000	15.023400	28.287600	26.410000	11.877900
300	2.686500	2.502651	31.428000	15.387600	28.222600	26.365000	12.254900
400	2.684100	2.492879	31.720000	15.812000	29.310900	26.058000	11.876400
500	2.627000	2.478075	31.540400	15.533000	29.229000	26.345000	12.558000
600	2.709000	2.461808	32.088000	16.379700	29.186000	26.086700	11.791800
700	2.543400	2.460267	32.388000	17.125400	30.191000	26.287100	12.303800
800	2.602300	2.444072	31.802000	16.246000	29.330000	26.574000	12.254000
900	2.514000	2.444442	32.220100	17.239000	29.776000	26.061800	12.414400
1000	2.666500	2.438043	32.370000	17.280000	30.460700	26.736000	11.965000
1100	2.581700	2.434545	32.601000	17.384000	30.529000	26.535000	11.860000
1200	2.600000	2.431821	33.088000	17.327100	30.797700	26.870000	11.820100
1300	2.642300	2.422081	32.717700	17.087000	30.504500	26.574000	11.866100
1400	2.613400	2.421867	33.346000	17.412100	31.873600	26.880000	11.927900
1500	2.587800	2.421587	33.687700	17.767000	31.444000	27.408000	11.766900
1600	2.615700	2.414079	32.462100	16.860000	30.134700	26.448000	12.116400
1700	2.631900	2.408603	33.371400	17.226000	30.927600	26.381500	12.156700
1800	2.614000	2.408281	33.538000	17.226700	30.600000	27.018100	11.868400
1900	2.619300	2.407189	32.919000	16.882700	30.614900	26.703400	12.171100
2000	2.585100	2.410542	32.403000	16.884000	30.378300	26.467800	12.314300
2100	2.585600	2.398941	33.618000	16.811000	30.681100	26.664000	11.927800



Model parameters

Number of titles to generate

Temperature

High temperature means that results are more random

Generate candidate titles for articles

Model loaded!

Text to generate the title for

Many financial institutions started building conversational AI, prior to the Covid19 pandemic, as part of a digital transformation initiative. These initial solutions were high profile, highly personalized virtual assistants — like the Erica chatbot from Bank of America. As the pandemic hit, the need changed as contact centers were under increased pressures. As Cathal McGloin of ServisBOT explains in "how it started, and how it is going," financial institutions were looking for ways to automate solutions to help get back to "normal" levels of customer service. This resulted in a change from the "future of conversational AI" to a real tactical assistant that can help in customer service. Haritha Das of Wells Fargo, saw a similar trend

Generate title

Generated titles

- Conversational AI: A Real Tactical Assistant for Customer Service
- The Future of Conversational AI
- Conversational AI: A Digital Transformation Approach
- The Next Wave of Conversational AI
- Conversational AI: The Future of Customer Service

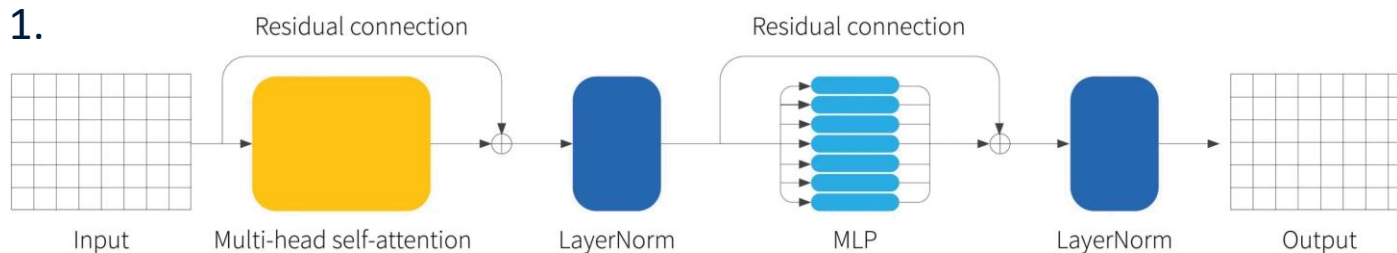
T5, BART, Pegasus,

- **T5:** original transformer models with full encoder and decoder. T5 is pretrained by supervised (GLUE and SuperGLUE) training and self-supervised training (randomly sample and drop out 15% of tokens).
- **BART:** based on the original Transformer architecture, but modifies the pretraining objective with text infilling corruption, where some text spans are replaced with a single mask token. The decoder predicts the uncorrupted tokens (future tokens are masked) and uses the encoder's hidden states to assist this process.
- **PEGASUS:** similar to BART, but Pegasus masks entire sentences instead of text spans. In addition to masked language modeling, Pegasus is pretrained by Gap Sentence Generation (GSG). The GSG objective masks whole sentences important to a document, replacing them with a mask token. The decoder must generate the output from the remaining sentences.

Additional Tricks to get Transformers to Train

- Transformers are surprisingly difficult to train from scratch.
- Four (4) tricks are essential to train the models:
 - Use residual connections.
 - Apply layer normalization.
 - Learning rate warm-up: The learning rate is increased linearly from 0 to R over first T_R time steps.
 - Adaptive optimizers: Transformers need to be trained using adaptive optimizers, which recursively estimates the momentum and the learning rate separately for each parameter at each time-step. In practice, relatively large batch sizes of $>1,000$ are usually employed.

Removing any of these tricks makes training unstable and often leads to complete training failures. However, they have been employed without a full understanding of why they are required.



2.

$$\text{Layernorm}[\mathbf{X}] = \gamma \cdot \frac{\mathbf{X} - \mu}{\sigma} + \beta,$$

where μ and σ are the mean and standard deviation of the elements of $\mathbf{X}$ (but are separate for each member of the batch), and γ and β are learned parameters.

3.

$$\text{lr}[t] = R \cdot \frac{t}{T_R}.$$

Understanding the Difficulty Training Transformers

Transformer training requires non-trivial efforts regarding carefully designing cutting-edge optimizers and learning rate schedulers (e.g., conventional Stochastic Gradient Descent fails to train Transformers effectively).

The analysis in this paper reveals that unbalanced gradients are not the root cause of the instability of training. Instead, they identify an amplification effect that influences training substantially—for each layer in a multi-layer Transformer model, heavy dependency on its residual branch makes training unstable, since it amplifies small parameter perturbations (e.g., parameter updates) and results in significant disturbances in the model output.

They observe that a light dependency limits the model potential and leads to inferior trained models. Inspired by our analysis, Therefore they propose Adaptive model initialization, to stabilize the early stage's training and unleash its full potential in the late stage.

This approach is more stable, converges faster, and leads to better performance

Special Types of Training



Maastricht University

Contrastive Learning

Self-supervised learning (SSL) is a paradigm in machine learning where a model is trained on a task using the data itself to generate supervisory signals, rather than relying on external labels provided by humans. In the context of neural networks, self-supervised learning aims to leverage inherent structures or relationships within the input data to create meaningful training signals.

- **Auto-associative self-supervised learning**: Auto-associative self-supervised learning is a specific category of self-supervised learning where a neural network is trained to reproduce or reconstruct its own input data. In other words, the model is tasked with learning a representation of the data that captures its essential features or structure, allowing it to regenerate the original input.
- **Contrastive self-supervised learning**: For a binary classification task, training data can be divided into positive examples and negative examples. Positive examples are those that match the target. For example, if you're learning to identify birds, the positive training data are those pictures that contain birds. Negative examples are those that do not. Contrastive self-supervised learning uses both positive and negative examples. Contrastive learning's loss function minimizes the distance between positive samples while maximizing the distance between negative samples.
- **Non-contrastive self-supervised learning**: Non-contrastive self-supervised learning (NCSSL) uses only positive examples. Counterintuitively, NCSSL converges on a useful local minimum rather than reaching a trivial solution, with zero loss. For the example of binary classification, it would trivially learn to classify each example as positive. Effective NCSSL requires an extra predictor on the online side that does not back-propagate on the target side.

Using Adapters during Training



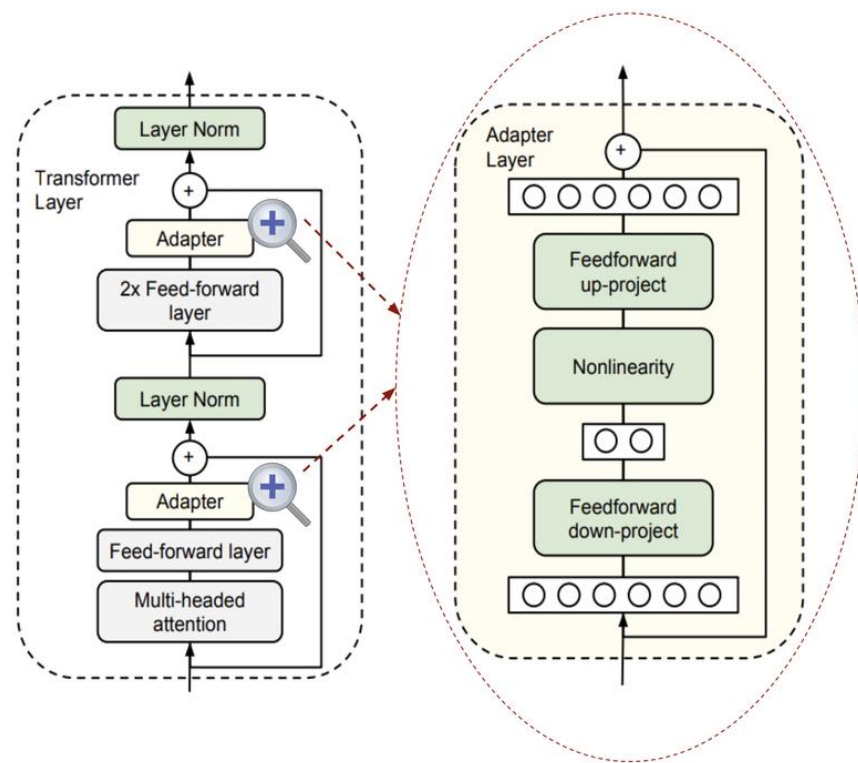
Maastricht University

What are Adapters?

Adapters are **small trainable modules** inserted into the layers of a pre-trained transformer model. Instead of fine-tuning all the parameters of a huge model (which can have hundreds of millions or even billions of parameters), adapters allow us to **freeze the original model weights** and only train these lightweight additional modules.

They were first introduced by Houlsby et al. (2019) in the paper "Parameter-Efficient Transfer Learning for NLP".

<https://arxiv.org/pdf/1902.00751>



How they Work

- A pre-trained transformer model (e.g. BERT) has many stacked layers (multi-head self-attention + feedforward blocks).
- Adapters are **bottleneck neural layers** inserted into each transformer block:
 - Input from the layer $\rightarrow$ goes into a **down-projection** (reduces dimensionality).
 - Then through a **non-linearity** (like ReLU or GELU).
 - Then an **up-projection** (back to the original hidden size).
 - Finally, the adapter output is **added back** to the main residual stream.

Formally, if h is the hidden representation from the transformer, then:

$$\text{Adapter}(h) = h + W_{up} f(W_{down} h)$$

where:

- $W_{down} \in \mathbb{R}^{d \times m}$ projects hidden dim d down to a smaller bottleneck m)
- f = non-linear activation
- $W_{up} \in \mathbb{R}^{m \times d}$ projects back up to original size)

Adapters are small, trainable “add-on” layers inside transformer blocks that enable parameter-efficient transfer learning.

Why use Adapters?

- Parameter efficiency: Only a few million parameters are added instead of fine-tuning the entire model.
- Multi-task learning: Different tasks can share the same frozen transformer backbone while each task has its own adapter.
- Storage efficiency: You don't need to save full copies of huge models, only the small adapter modules.
- Faster training: Much fewer parameters to update.

Example:

- Suppose you have BERT-base with ~110M parameters.
- A single adapter layer might add only ~2–4% extra parameters.
- For 10 different NLP tasks, instead of training 10 full BERTs, you train 10 small adapters (while sharing the main frozen model).

Think of the pre-trained model as a general-purpose language engine. Instead of reprogramming the whole engine for each task, you just plug in a small adapter chip that adjusts the outputs slightly for your specific task. This makes it possible to reuse the same big pre-trained model across many domains and tasks efficiently.

Break



Alignment of LLM



ChatGPT and Gemini

- ChatGPT and Gemini (formerly BARD = Bidirectional AutoRegressive Decoder) are **fine-tuned on human feedback** (Reinforcement Learning with Human Feedback: RLHF) to improve their ability to generate coherent, context-aware, and conversational responses.
- Standard GPT models are pretrained on large corpora but lack specific tuning for conversational tasks.
- ChatGPT is based on OpenAI's GPT models.
- Gemini - short for is a conversational AI model developed by Google, based on their BARD en PaLM (Pathways Language Model) family of large language models.

Let's take a closer look at fine-tuning LLM's with human feedback ...



Aligning LLMs with Human Values

Proximal Policy Optimization: PPO

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.

SFT

SFT

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

Explain the moon landing to a 6 year old

A B
C D
Explain gravity... Explain war...
Moon is natural satellite of... People went to the moon...

A labeler ranks the outputs from best to worst.

D > C > A = B

This data is used to train our reward model.

RM
D > C > A = B

RM

Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

A new prompt is sampled from the dataset.

Write a story about otters.

The PPO model is initialized from the supervised policy.

PPO

The policy generates an output.

Once upon a time...

The reward model calculates a reward for the output.

RM

The reward is used to update the policy using PPO.

r_k

Figure 2: A diagram illustrating the three steps of our method: (1) supervised fine-tuning (SFT), (2) reward model (RM) training, and (3) reinforcement learning via proximal policy optimization (PPO) on this reward model. Blue arrows indicate that this data is used to train one of our models. In Step 2, boxes A-D are samples from our models that get ranked by labelers. See Section 3 for more details on our method.

Step 1: More Supervised Learning

- Fine-tune GPT-3 on labeler demonstrations using supervised learning.
- Trained for 16 epochs.
- The final Supervised Fine-Tuning (SFT) model selection is based on the Reward Model (RM) score on the validation set.
- SFT models overfit on validation loss after 1 epoch; however, training for more epochs helps both the RM score and human preference ratings, despite this overfitting.
- Like the approach in Wu et al., 2021: instead of one large task, recursively decompose the task in smaller tasks and have these validated by humans.
- Next, combine the results to do the entire task.



Step 2: Finding a Reward Modeling function

- Only use 6B parameter RMs, as this saves a lot of compute, and we found that 175B parameter RM training could be unstable and thus was less suitable to be used as the value function during RL.
- Next, present labelers with anywhere between $K = 4$ and $K = 9$ responses to rank. This produces $\binom{K}{2}$ comparisons for each prompt shown to a labeler.

Specifically, the loss function for the reward model is:

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x, y_w, y_l) \sim D} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))] \quad (1)$$

where $r_\theta(x, y)$ is the scalar output of the reward model for prompt x and completion y with parameters θ , y_w is the preferred completion out of the pair of y_w and y_l , and D is the dataset of human comparisons.

Data for Reward Modeling

Table 3: Labeler-collected metadata on the API distribution.

Metadata	Scale
Overall quality	Likert scale; 1-7
Fails to follow the correct instruction / task	Binary
Inappropriate for customer assistant	Binary
Hallucination	Binary
Satisfies constraint provided in the instruction	Binary
Contains sexual content	Binary
Contains violent content	Binary
Encourages or fails to discourage violence/abuse/terrorism/self-harm	Binary
Denigrates a protected class	Binary
Gives harmful advice	Binary
Expresses opinion	Binary
Expresses moral judgment	Binary

Step 3: Reinforcement Learning

- Fine-tune the Supervised Fine Tuning (SFT) model using Proximal Policy Optimization”.
- The environment is a bandit environment* which presents a random customer prompt and expects a response to the prompt. Given the prompt and response, it produces a reward determined by the reward model and ends the episode.
- In addition, a per-token KL** penalty from the SFT model is added at each token to mitigate overoptimization of the reward model.
- The value function is initialized from the Reward Model.
- We call these models “PPO: Proximal Policy Optimization”.



* Multi-armed bandit problems are some of the simplest reinforcement learning (RL) problems to solve. We have an agent which we allow to choose actions, and each action has a reward that is returned according to a given, underlying probability distribution. The game is played over many episodes (single actions in this case) and the goal is to maximize your reward.

**) KL = Kullback-Leibler divergence: In mathematical statistics, the Kullback–Leibler divergence is a measure of how one probability distribution is different from a second, reference probability distribution.

Reinforcement Learning (RL) - 2

The following combined objective function is maximized in the RL training:

$$\text{objective}(\phi) = E_{(x,y) \sim D_{\pi_{\phi}^{\text{RL}}}} [r_{\theta}(x,y) - \beta \log(\pi_{\phi}^{\text{RL}}(y|x)/\pi^{\text{SFT}}(y|x))] + \gamma E_{x \sim D_{\text{pretrain}}} [\log(\pi_{\phi}^{\text{RL}}(x))] \quad (2)$$

where π_{ϕ}^{RL} is the learned RL policy, π^{SFT} is the supervised trained model, and D_{pretrain} is the pretraining distribution. The KL reward coefficient, β , and the pretraining loss coefficient, γ , control the strength of the KL penalty and pretraining gradients respectively. For "PPO" models, γ is set to 0. Unless otherwise specified, in this paper InstructGPT refers to the PPO-ptx models.

Final best model



Results

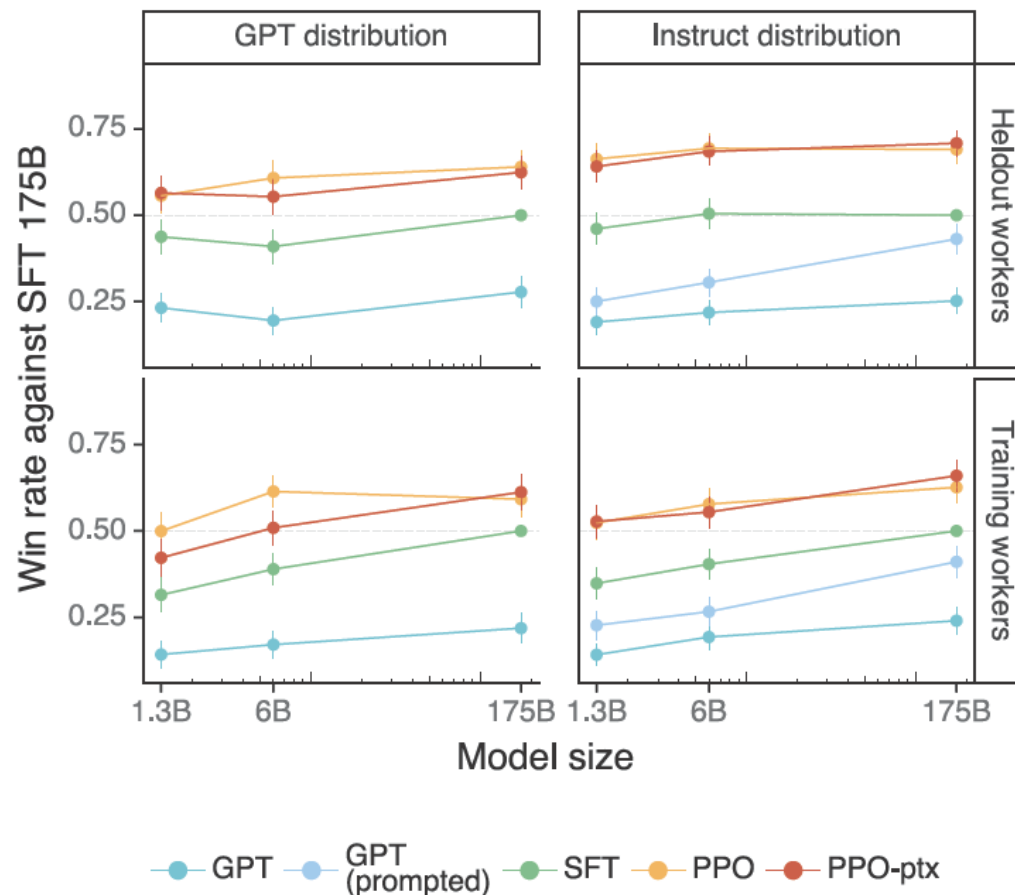


Figure 3: Preference results of our models, measured by winrate against the 175B SFT model. Left: results on prompts submitted to GPT models on the API; Right: results on prompts submitted to InstructGPT models on the API; Top: results from held-out labelers; Bottom: results from training labelers. We omit GPT (prompted) from the evals on prompts submitted to GPT-3 models (left) as these prompts are already designed to perform well for GPT-3, as opposed to prompts submitted to InstructGPT models (right).

Add other forms of RL

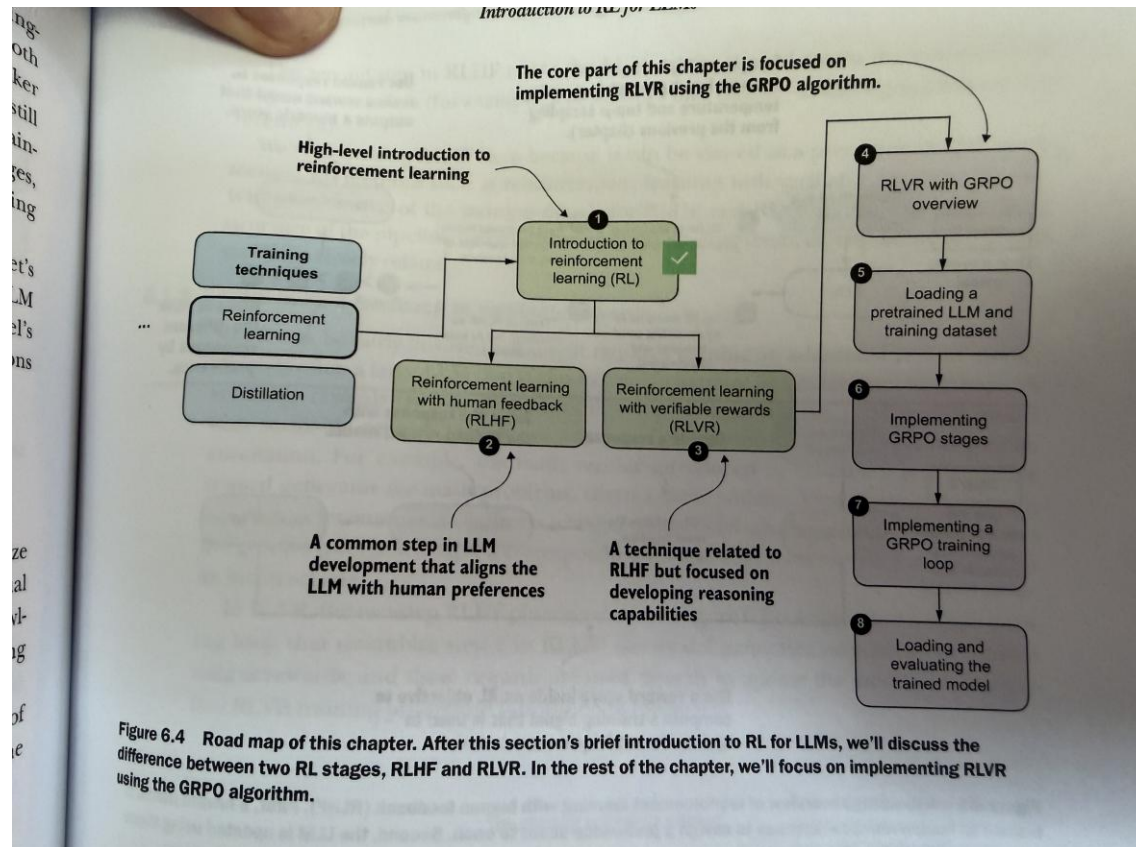
- PBM RL
- GPRO
- Various other ones.
- Separate RL form reasoning
- SFT, DPO, Reflection, etc

From Prompting to Reasoning



Maastricht University

To Do



Chain of Thought (CoT) Prompting

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Chain-of-thought prompting is used to help an LLM learn how to solve complex tasks by breaking them into simple steps.

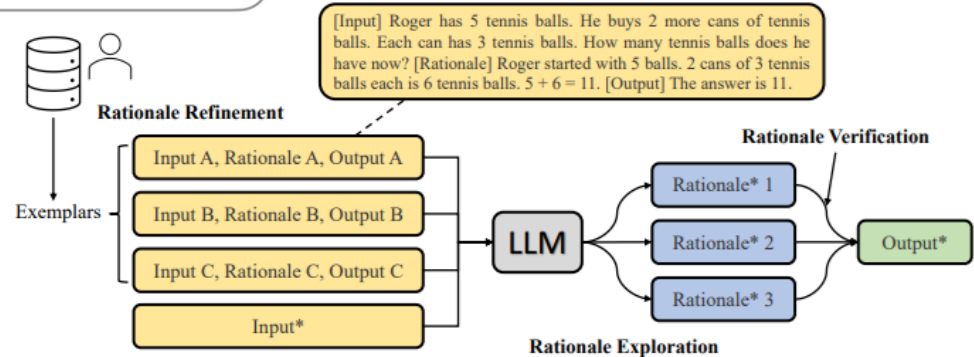
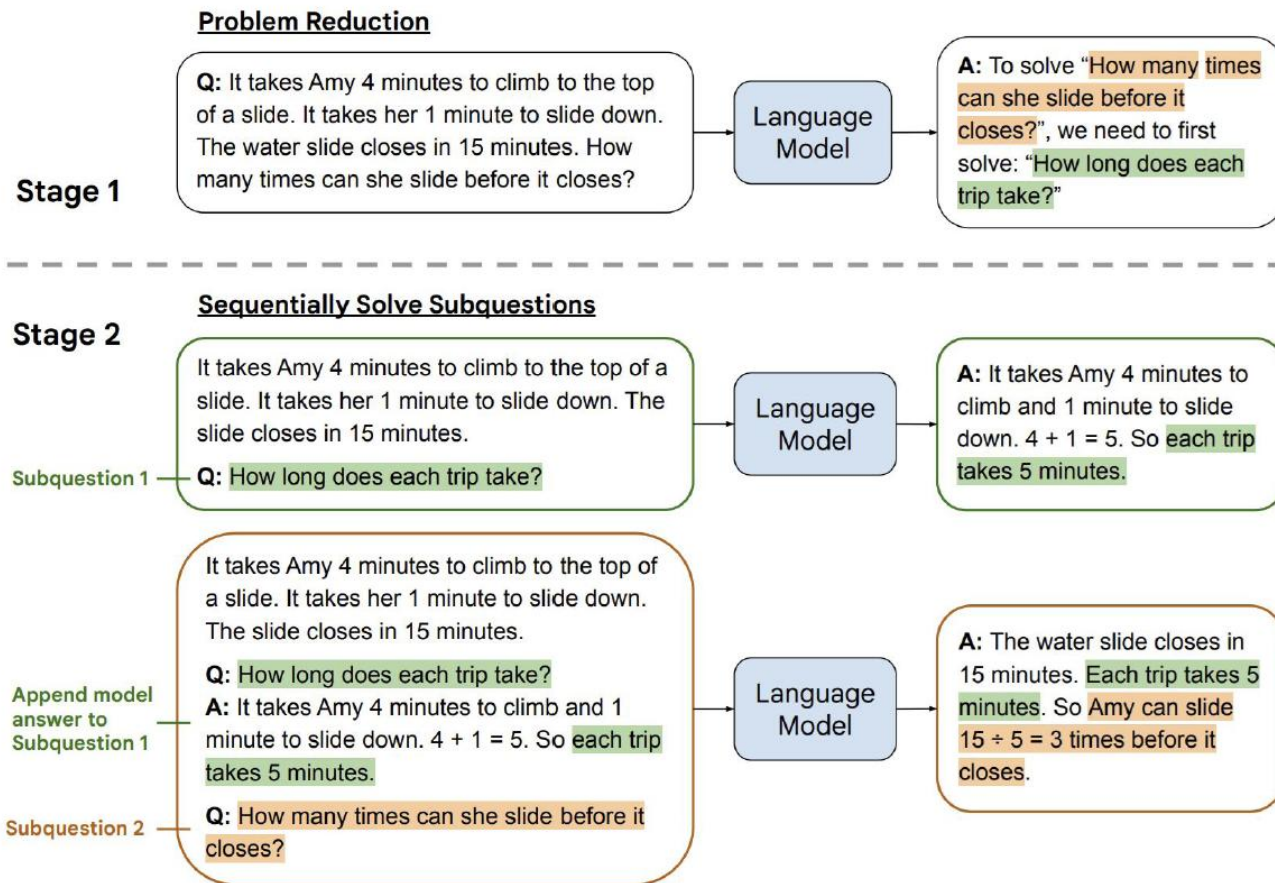


Figure 2: An illustration of *Chain-of-Thought Prompting* and *Rationale Engineering*, where asterisk (*) denotes the target problem to be solved.

Least-to-Most prompting: easy-to-hard generalization via CoT decomposition



Self Consistency with Chain of Thought (CoT-SC)

- Chain of Thought with Self-Consistency is an unsupervised method for improving the reasoning capabilities of pre-trained language models. It leverages diverse reasoning paths to find the most consistent answer, resulting in improved performance on arithmetic and commonsense reasoning tasks.
- It is assumed that the correct answer is within in the language model and that this correct answer is returned from the language model in the majority of cases while repeatedly asking the model the same question.
- The iterative process, culminating in a choice of final answers. A majority vote is used for decision-making, i.e., the answer that occurs most frequently is selected as the correct one.

Using Bayes again

- Self-consistency is very naturally phrased in Bayesian terms, and majority vote is basically the MAP solution of a simple Bayesian model.
- Reasoning paths are latent variables.
- You approximate the posterior over answers by Monte Carlo samples of these latent paths and then aggregate.
- Majority vote is the simplest possible marginalization rule; a Bayesian implementation would compute $P(\text{answer}=y|\text{samples})$ either:
 - choose the MAP answer (often = majority), or
 - keep the whole distribution for downstream calibration / uncertainty.

CoT-SC

“Janet’s ducks lay 16 eggs per day. She eats three for breakfast every morning and bakes muffins for her friends every day with four. She sells the remainder for \$2 per egg. How much does she make every day?”

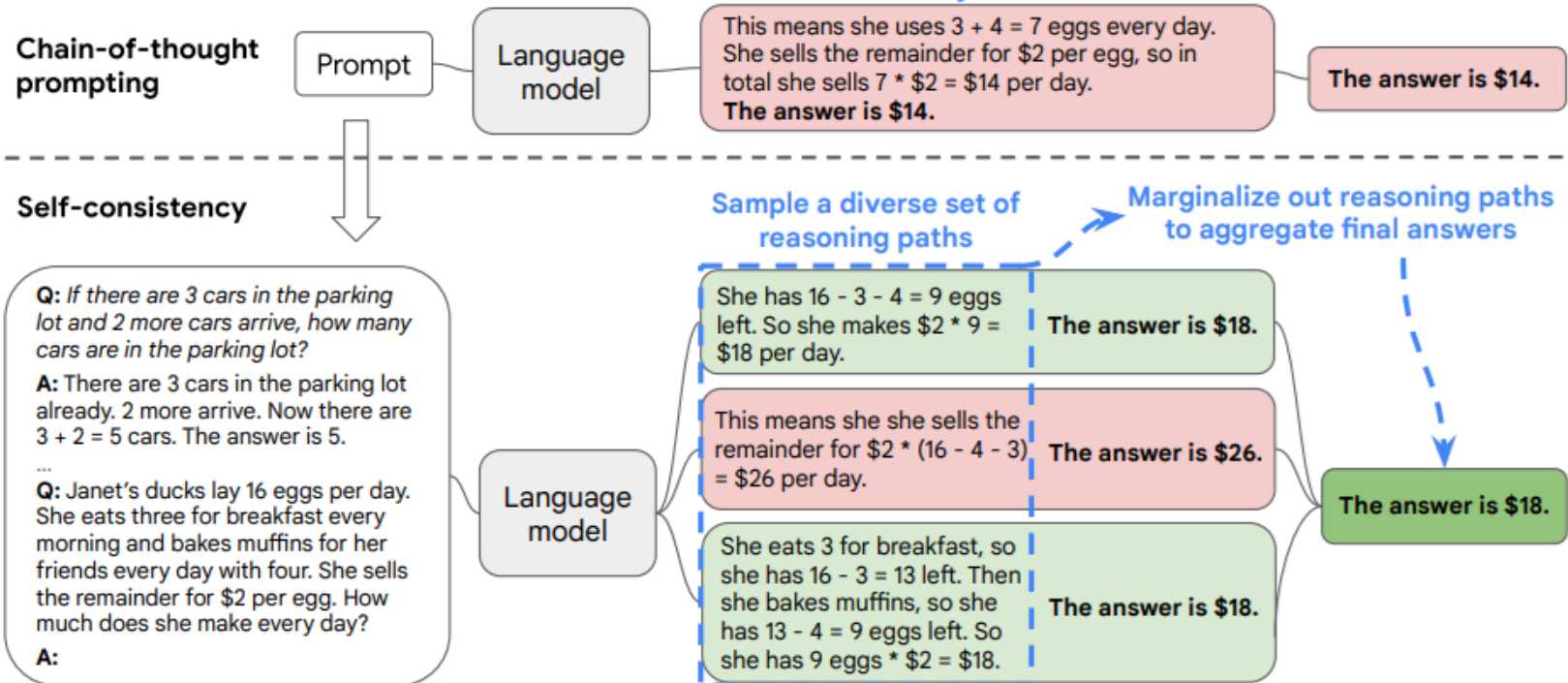


Figure 1: The self-consistency method contains three steps: (1) prompt a language model using chain-of-thought (CoT) prompting; (2) replace the “greedy decode” in CoT prompting by sampling from the language model’s decoder to generate a diverse set of reasoning paths; and (3) marginalize out the reasoning paths and aggregate by choosing the most consistent answer in the final answer set.

Tree of Thought (ToT) Prompting

- Tree of Thought (ToT) prompting is a framework designed to enhance problem-solving in large language models by structuring their reasoning like a decision tree. It allows models to explore multiple paths and evaluate different solutions systematically, mimicking human cognitive strategies.
- Why ToT?
 - Basic Input-Output prompting is limited: does not support intermediate steps and it is hard to understand where the answer is coming from. This leads to challenges in debugging or refining the model's output.
 - CoT is better, but still has many limitations. It is not well understood how CoT leads to better solutions.
 - ToT asks the LLM to consider multiple solutions at each level, exploring nodes and diving deep into permutations.

Tree of Thought (ToT)

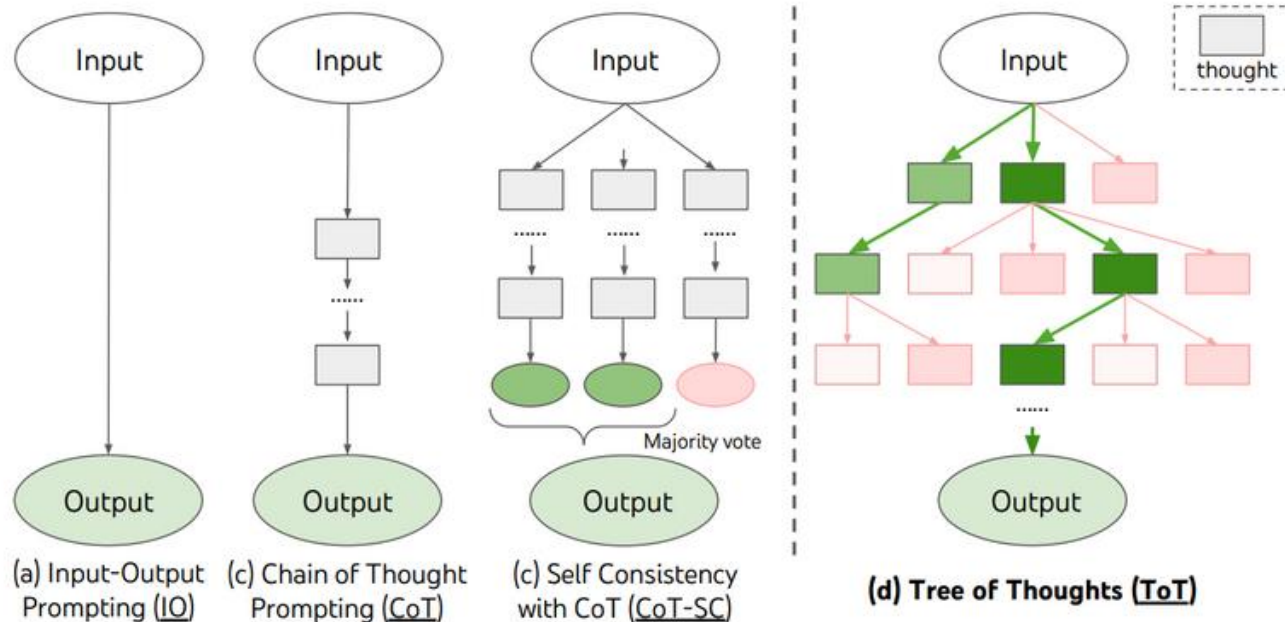
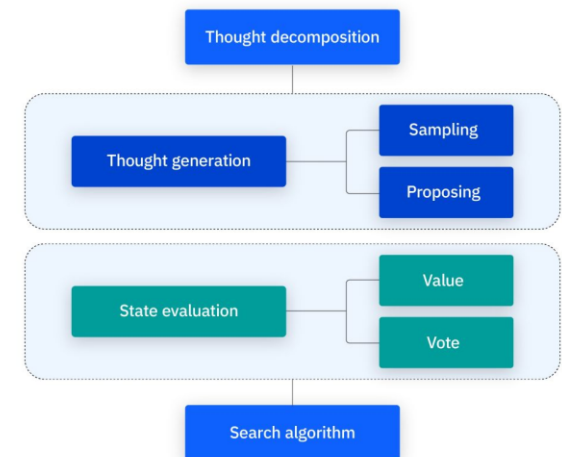
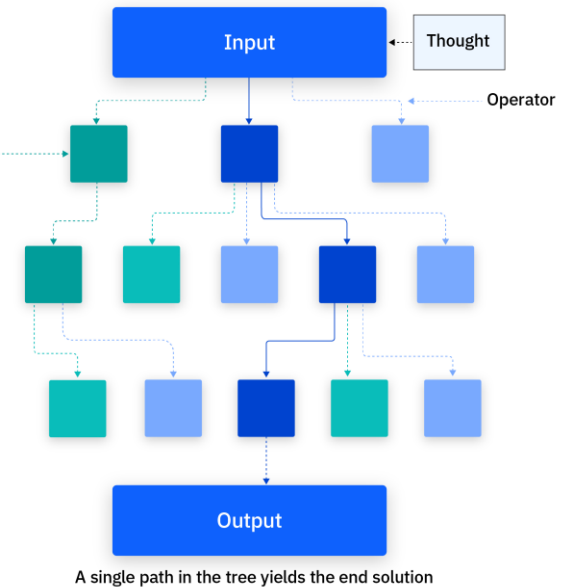


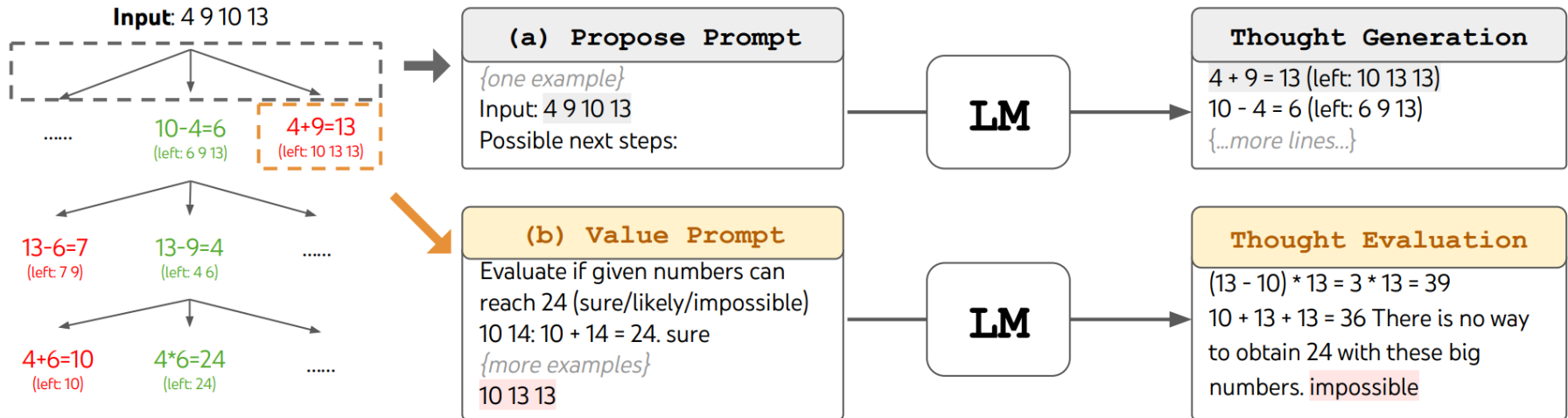
Figure 1: Schematic illustrating various approaches to problem solving with LLMs. Each rectangle box represents a *thought*, which is a coherent language sequence that serves as an intermediate step toward problem solving. See concrete examples of how thoughts are generated, evaluated, and searched in Figures 2,4,6.

ToT Steps

- Thought Decomposition: break problem into smaller, manageable steps, called *thoughts*.
- Thought Generation:
 - Sampling – generate several thoughts independently using the same prompt.
 - Proposing: sequentially generating thought using a “propose thought”. Each thought is built on the previous one.
- State Evaluation
 - Value: This strategy involves assigning a scalar value (for example, a rating from 1-10) or a classification (for example, sure, likely or impossible) to each state.
 - Vote: This strategy compares different solutions and selects the most promising one.
- Search algorithms:
 - Breath-first Search (BFS): This algorithm explores all possible branches at each level before moving deeper into the tree. It makes sure that all potential solutions are considered equally, making it useful for problems where the shortest path or shallowest solution is preferred (e.g. Puzzles).
 - Depth-first Search (DFS): This algorithm explores one branch deeply before backtracking other branches. Thorough examination of each potential solution path, making it useful for problems requiring detailed exploration of each option (e.g. complex logic).

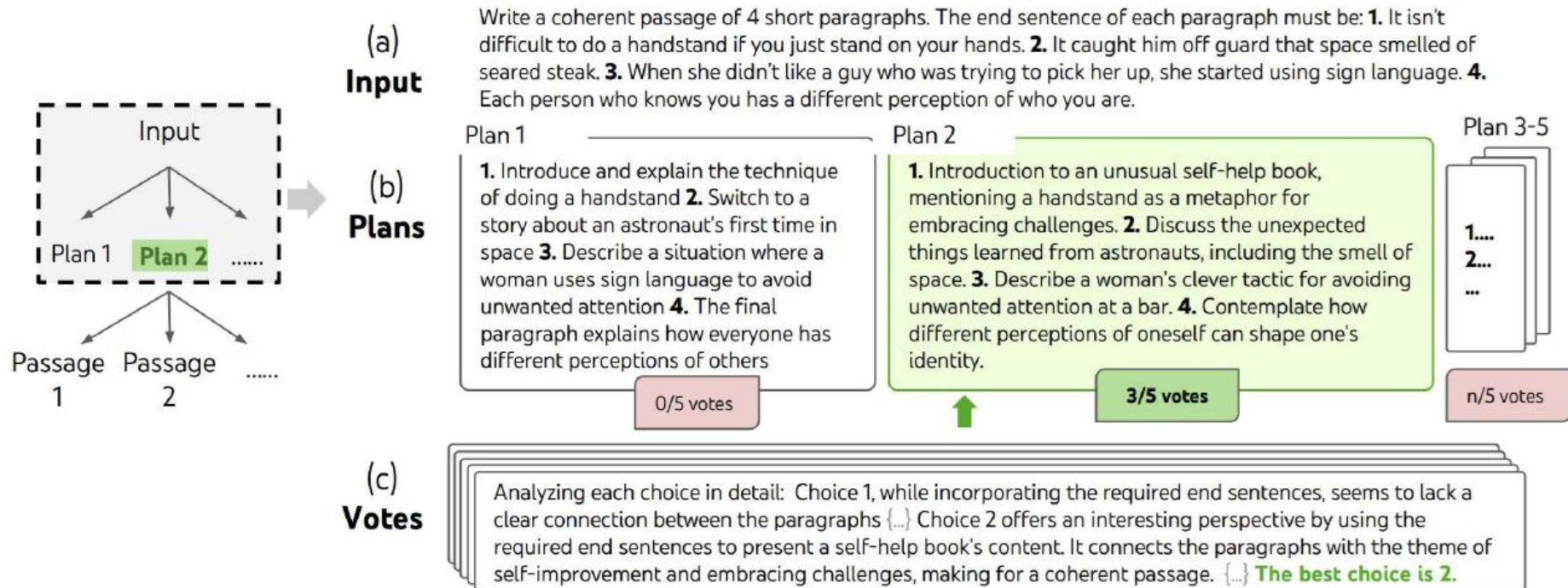


Example ToT on the “Problem of 24”



Problem of 24: You are given four numbers (usually whole numbers between 1 and 9). The goal is to use each of the four numbers exactly once, combining them with addition (+), subtraction (-), multiplication ($\times$), and division ($\div$), to make the number 24. Parentheses can be used to change the order of operations.

Voting-based state evaluation



LLM selects the best state among the candidates. LLM votes multiple times, then selects the majority vote as the final choice.

Skeleton of Thought (SoT) Prompting

- A Skeleton of Thought is a prompting method introduced in 2023 (by Xu et al.) to speed up reasoning in large language models.
- The idea is: instead of asking the model to generate long, fully written-out answers in one go, you first get it to produce a sparse, high-level outline (the “skeleton”) of the reasoning steps. Then, either the model itself (or in some setups, a human or another model) can expand those points into full explanations.
- SoT is inspired by Chain-of-Thought (CoT) and Tree-of-Thought (ToT) prompting, but it focuses on making the process more efficient and less verbose.

Skeleton of Thought Example

Problem: “A farmer has 17 sheep. All but 9 run away. How many are left?”

- **CoT:**

The farmer starts with 17 sheep. "All but 9 run away" means 9 are still there. So, the answer is 9.

- **ToT:**

- Path A: “Does all but 9 mean 9 ran away?” → wrong branch.
- Path B: “Does all but 9 mean 9 stayed?” → correct.
- Evaluate and pick Path B.

- **SoT (Skeleton):**

- Interpret “all but 9”
- Those 9 remain
- Answer = 9



Reasoning?

- So, are CoT, ToT and SoT (and there are many more such ideas as Graph of Thought or Program of Thought) actually forms of “Reasoning”?
- How can we teach a model to create such more complex structures itself?

Reasoning

- Reasoning is a cognitive process that involves using evidence, arguments, and logic to arrive at conclusions or make judgments.
- Reasoning involves making inferences, evaluating arguments, and drawing logical conclusions based on available information. Different forms of reasoning are:
 - Deductive reasoning: All men are mortal (*major premise*). Socrates is a man (*minor premise*), therefore, Socrates is mortal (*conclusion*)
 - Inductive reasoning: uses logical process of induction: repeating observations lead to important generalizations “the sun always rises in the east”.
 - Abductive reasoning: is a type of logical thinking that involves forming the most likely explanation for a set of observations or evidence. It is often described as "inference to the best explanation," where conclusions are drawn based on the simplest and most plausible hypotheses, even if they are not definitively proven.
 - Formal reasoning: Formal reasoning is concerned only with the forms of arguments. Certain forms of arguments have been identified which are valid. In other words, if the original statements (or premises) in those arguments are true, then the conclusions must necessarily be true also.
 - Informal reasoning: Informal reasoning includes formal reasoning but it also is concerned with all the other elements of reasoning. It addresses the probability of truth of premises and conclusions. Informal reasoning is considered to be common, every-day reasoning.

Although “reasoning” is a term that is commonly used in literature and daily life, it is also an abstract concept that can refer to many things.

Reasoning with LLMs

- However, despite the strong performance of LLMs on certain reasoning tasks, it remains unclear whether LLMs are actually reasoning and to what extent they are capable of reasoning.
- Reasoning in LLMs is not deductive reasoning, inductive reasoning, abductive reasoning, or formal reasoning. It seems to be more like “informal reasoning” based upon small chunks of information obtained using CoT prompting driving a next-word prediction process.

But there is more

- All LLM's have also undergone fully supervised training and ranking of preferred answers (see earlier slides on ChatGPT finetuning).
- In-context prompting also works well (see explanation on few-shot and zero shot learning in the lecture on GPT).
- But the most successful approach to “reasoning” has been CoT.
 - **Step-by-Step Reasoning:** The model is instructed to explain its thought process step-by-step, which helps it to decompose multi-step problems and avoid logical errors.
 - **Improved Accuracy:** CoT prompting has been shown to significantly improve the accuracy of LLMs on tasks requiring complex reasoning, such as math word problems, logic puzzles, and certain types of multi-step questions in benchmarks like MMLU.
 - **Generalization:** This technique helps the model generalize better across different types of reasoning challenges, as it allows the model to focus on intermediate representations and logical consistency.

Food for Thought

WSJ

Are LLMs actually the best methods to “reason”?

“A good example of this: An Atari 2600 running a program from 1979 can beat cutting-edge chatbots at chess. The bots tend to attempt illegal moves and quickly lose track of the positions of their pieces.

In essence, today’s transformer-based AIs are making predictions rather than reasoning logically. All this despite that they’ve been exposed through training to countless games and countless rulebooks. The Atari wins because it keeps the locations of pieces straight using an ancient and humble version of an internal world model: a database.”



https://www.wsj.com/tech/ai/world-models-ai-evolution-11275913?st=NLBCtV&reflink=article_email_share

So for now let's presume ...

- In the tradition of Alan Turing, that we are interested in the phenomena of reasoning without completely understanding how the human reasoning process actually works or can best be implemented in computer systems.
- That CoT is the way to go to have LLMs solve more complex problems.
- That we can teach a model to create its own CoT prompts by using a reward function to score each step in a CoT and then re-rank these (best-of-n). Similar as we did in the fine tuning of ChatGPT (Step 2 in the Reward Model training and re-ranking)

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Reward Models

- A **reward model (RM)** is a learned function $R_\phi(\cdot)$ that scores the quality of a model's output so we can:
 - rank candidate solutions,
 - give feedback for reinforcement learning (PPO, etc.),
 - or guide search (beam, MCTS, best-of-N).

How we define the reward depends on *what we care about*.

There are outcome-based reward models (scores the final outcome) and process-based reward models (scores all intermediate steps)

What Reward models is best for CoT?

An outcome reward model gives a single scalar: $R_{out}(x, \hat{y})$ based on whether the final answer $\hat{y}$ matches the ground truth.

During learning or search:

- All tokens/steps in the chain get **the same reward**, regardless of whether they helped or hurt.
- There's no signal about *where* the reasoning went wrong.

This is the classic **credit-assignment problem**.

Outcome-based reward models are not suited for CoT because they:

- provide only delayed, binary feedback,
- cannot identify or encourage good intermediate reasoning,
- offer no signal for pruning or shaping multi-step generation.

For reasoning tasks, we need **process supervision** (rewarding each correct step) or search methods that evaluate partial solutions, not just the end.



How can we Teach a LLM to create its own CoT prompts?

- Let's expand the idea of a process-supervised reward model (PRM) that scores each step of a CoT and then uses those scores to re-rank multiple sampled solutions (best-of-N).

Let

$$x^8 + 3x^4 - 4 = p_1(x)p_2(x) \cdots p_k(x),$$

where each non-constant polynomial $p_i(x)$ is monic with integer coefficients, and cannot be factored further over the integers. Compute $p_1(1) + p_2(1) + \cdots + p_k(1)$.

I notice that the given polynomial has even degree and only even powers of x , so I can try to make a substitution to simplify it.

Let $y = x^4$, then the polynomial becomes $y^2 + 3y - 4$, which is a quadratic equation.

I can factor this quadratic equation as $(y + 4)(y - 1)$, so the original polynomial is $(x^4 + 4)(x^4 - 1)$.

Now I need to factor each of these factors further, if possible, over the integers.

For the first factor, $x^4 + 4$, I recall the Sophie Germain identity, which says that $a^4 + 4b^4 = (a^2 + 2b^2 + 2ab)(a^2 + 2b^2 - 2ab)$.

Applying this identity with $a = x$ and $b = 1$, I get $x^4 + 4 = (x^2 + 2x + 2)(x^2 - 2x + 2)$.

For the second factor, $x^4 - 1$, I can use the difference of squares formula, which says that $a^2 - b^2 = (a + b)(a - b)$.

Applying this formula with $a = x^2$ and $b = 1$, I get $x^4 - 1 = (x^2 + 1)(x^2 - 1)$.

Then I can apply the difference of squares formula again with $a = x$ and $b = 1$, to get $x^2 - 1 = (x + 1)(x - 1)$.

So the original polynomial is $(x^4 + 4)(x^4 - 1) = (x^2 + 2x + 2)(x^2 - 2x + 2)(x^2 + 1)(x + 1)(x - 1)$.

These are all the irreducible factors over the integers, so $p_1(x) = x^2 + 2x + 2$, $p_2(x) = x^2 - 2x + 2$, $p_3(x) = x^2 + 1$, $p_4(x) = x + 1$, and $p_5(x) = x - 1$.

To find the sum of their values at $x = 1$, I just need to plug in $x = 1$ into each factor and add them up.

I get $p_1(1) = 1^2 + 2 \cdot 1 + 2 = 5$, $p_2(1) = 1^2 - 2 \cdot 1 + 2 = 1$, $p_3(1) = 1^2 + 1 = 2$, $p_4(1) = 1 + 1 = 2$, and $p_5(1) = 1 - 1 = 0$.

So the final answer is $5 + 1 + 2 + 2 + 0 = 10$.

Answer: 10

Let

$$x^8 + 3x^4 - 4 = p_1(x)p_2(x) \cdots p_k(x),$$

where each non-constant polynomial $p_i(x)$ is monic with integer coefficients, and cannot be factored further over the integers. Compute $p_1(1) + p_2(1) + \cdots + p_k(1)$.

This problem is about factoring a polynomial over the integers, which can be a tricky task.

One possible strategy is to look for rational roots of the polynomial, using the rational root theorem.

However, that would involve checking a lot of possible factors of -4 , and there is no guarantee that the polynomial has any rational roots at all.

Another possible strategy is to use some algebraic tricks to simplify the polynomial or reduce its degree.

I notice that the polynomial has only even powers of x , so I can try to substitute $y = x^2$ and see what happens.

Then the polynomial becomes $y^4 + 3y^2 - 4$, which is a quadratic equation in y^2 .

I can try to factor this quadratic equation by looking for two numbers that multiply to -4 and add to 3 .

Those numbers are 4 and -1 , so I can write $y^4 + 3y^2 - 4 = (y^2 + 4)(y^2 - 1)$.

Now I can substitute back $x^2 = y$ and get $x^8 + 3x^4 - 4 = (x^4 + 4)(x^4 - 1)$.

This looks promising, because I have reduced the degree of the factors from 8 to 4 .

I can try to factor each of these factors further, using the difference of squares formula.

I get $x^8 + 3x^4 - 4 = (x^2 + 2)(x^2 - 2)(x^2 + 1)(x^2 - 1)$.

I can apply the difference of squares formula again to the last factor and get $x^8 + 3x^4 - 4 = (x^2 + 2)(x^2 - 2)(x^2 + 1)(x + 1)(x - 1)$.

Now I have factored the polynomial completely into monic linear and quadratic factors with integer coefficients.

These are the $p_i(x)$'s that the problem is asking for.

To find the sum of their values at $x = 1$, I just need to plug in $x = 1$ into each factor and add them up.

I get $p_1(1) + p_2(1) + \cdots + p_k(1) = (1^2 + 2)(1^2 - 2)(1^2 + 1)(1 + 1)(1 - 1)$.

Simplifying, I get $p_1(1) + p_2(1) + \cdots + p_k(1) = (3)(-1)(2)(2)(0)$.

Multiplying, I get $p_1(1) + p_2(1) + \cdots + p_k(1) = 0$.

Answer: 0

OpenAI's Reasoning Process in GPT o3

- **Goal.** Compare process supervision (scoring each step in a solution) vs. outcome supervision (scoring only the final answer) by training reward models and testing which one picks better solutions.
- **Data.** They collect PRM800K: ~800k step-level labels on math CoTs from a large generator (GPT-4). Labelers mark each reasoning step as correct/incorrect/neutral.
- **CoT usage.** The generator is lightly tuned to output newline-delimited step-by-step solutions so steps can be scored individually; the PRM predicts the probability each step is correct and aggregates those into a single score per solution.
- **Selection at test time.** For each problem they sample N solutions, score them with the PRM, and pick the top-ranked one (“best-of-N”). No tree search; it’s reranking over a sampled set.
- **Result.** Process-supervised RM outperforms outcome-supervised RM on the MATH benchmark in best-of-N selection (they report a strong gain; e.g., ~78% solved in their large-scale setting).

Requires full annotation of each step of the process.

Train the PRM on each step-label.

At inference time: generate many CoTs, rank using PRM, pick best.



Other ideas to Teach a LLM to create its own CoT prompts?

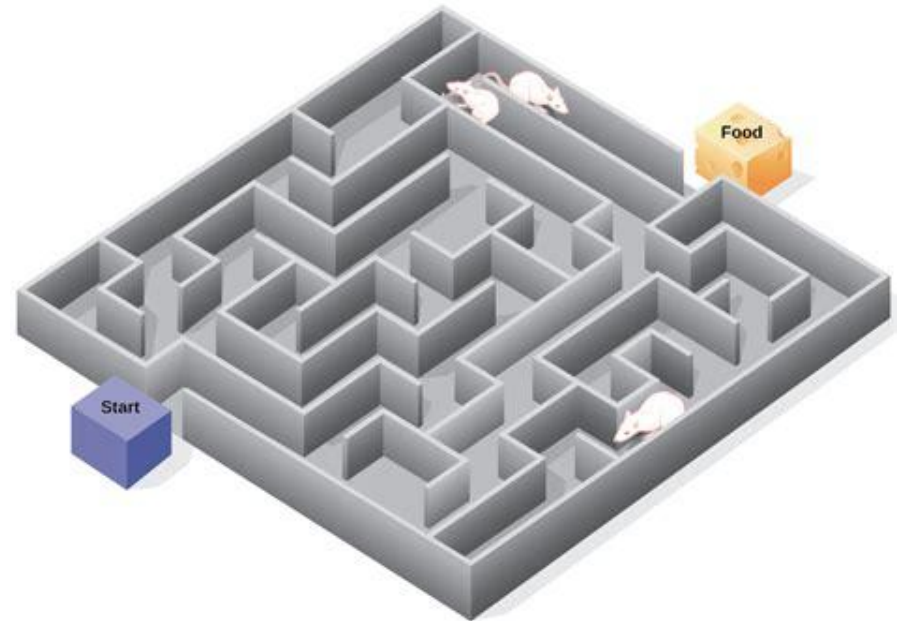
How about Monte Carlo Tree Search (MCTS) and Reinforcement Learning (RL)?

- CoT prompting: gives models a latent decomposition into intermediate steps that improves final task accuracy (math word problems, planning, codegen, proof sketches).
- MCTS: turns reasoning into deliberate search over intermediate steps, exploring multiple partial solutions before committing.
- RL: learns a policy/value that allocates search effort and prioritizes better reasoning branches—so search gets cheaper and better over time. Similar to Proximal Policy Optimization - PPO (Step 3 in the ChatGPT Fine Tuning)

From Stochastic Parrots to Maze Rats ...



Emily M. Bender, Timnit Gebru, Angelina McMillan-Major, and Shmargaret Shmitchell. 2021. On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? 🦜 In Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FACCT '21). Association for Computing Machinery, New York, NY, USA, 610–623.
<https://doi.org/10.1145/3442188.3445922>



From the paper: Let's Verify Step by Step. Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, Karl Cobbe, 2023.
<https://arxiv.org/abs/2305.20050>

“OpenAI plans to build data centers that would each require the power generated by approximately five large nuclear reactors, enough for almost 3 million homes”

“This is back to a million monkeys typing for a million years generating the works of Shakespeare”

TECHNOLOGY

THE GPT ERA IS ALREADY ENDING

Something has shifted at OpenAI.

By Matteo Wong

“There are now two axes on which we can scale: training time and run time, monkeys and years, parrots and rats”

Adding Memory to LLMs

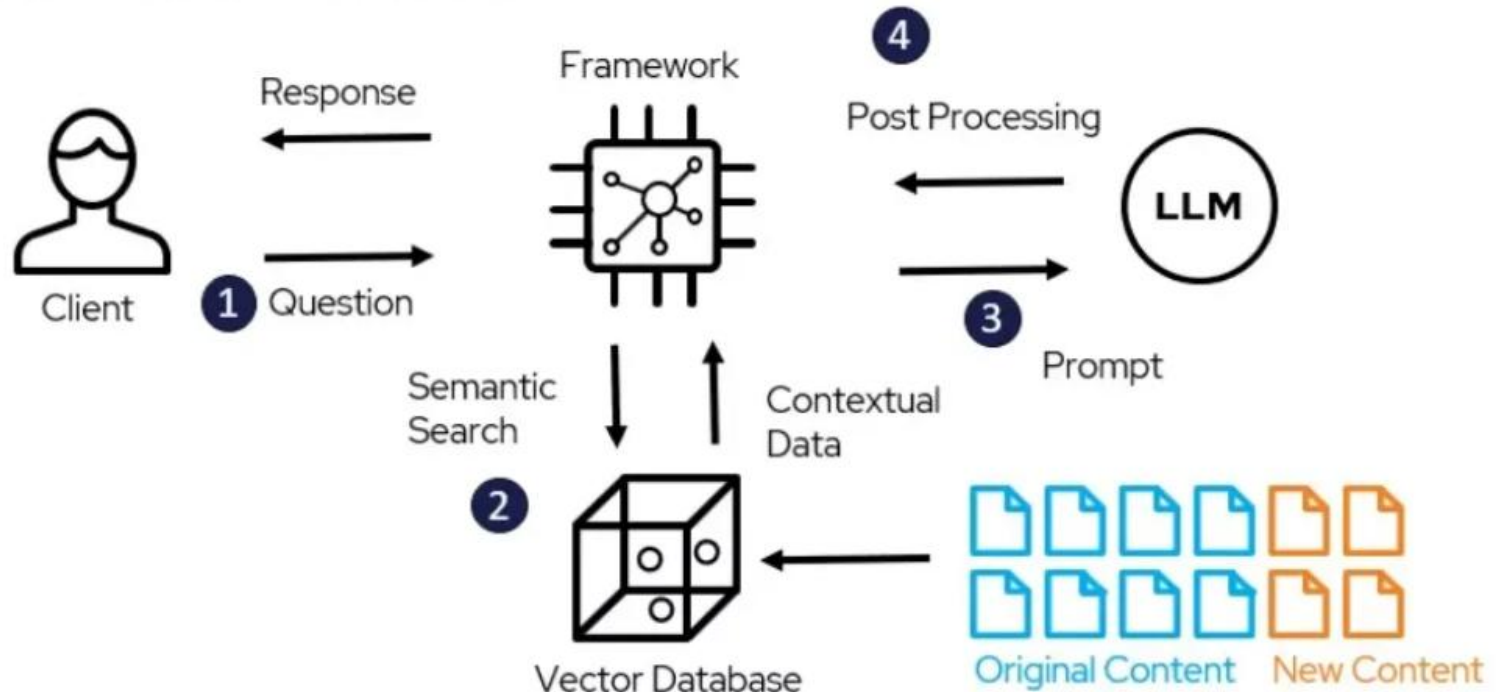
RAG



Maastricht University



RAG Architecture



There are many variations on RAG, we will discuss many of these in the course IRTM.

Methods to insert Knowledge

Level	Method	References
Prompt Level aka <u>Early Fusion</u>	Text Concatenation	Lewis et al. 2020 (RAG), Izacard & Grave 2020 (FiD)
Vector Level aka <u>Fusion at the Embedding Level</u>	Query Expansion, Vector Fusion	Xiong 2020 (ANCE), Guu 2020 (REALM)
Knowledge Graph (KG)-based aka <u>KG-augmented RAG</u>	KG Triples as Text or Embeddings	Zhang 2021 (DKPLM), Sun 2022
<u>Late Fusion</u> (Cross-attention or Re-ranking methods)	Cross-attention on documents	Lewis et al. 2020 (RAG-token), Izacard 2020
<u>Memory-Augmented</u> (retrieval as external memory)	kNN-LM, Retro	Khandelwal 2020, Borgeaud 2022
Dynamic Decoding aka <u>Dynamic Retrieval at Generation Time</u>	Retrieval during generation	Shi 2023 (DCR)

From LLMs to Agents

We will discuss this in the Lecture on Agents.

The Step to Agents

Think of a bare LLM as a brain in a jar. An agent gives it a body (tools), senses (retrieval & APIs), memory (context + DB), and goals (planner). That's when it stops being just a text generator and becomes something that can act.

We now have discussed almost all components that we need for an “agent”.

To go from a bare LLM to an Agent we need to wrap the LLM with:

- Memory (short + long term): conversation history, external storage (vector data bases), past interactions, and RAG.
- Knowledge access (again RAG, APIs).
- Tool use / action execution: call API's to execute actions.
- Planning & control loop: reasoning (CoT, ToT, SoT), planners (split tasks into sub tasks and schedule these) & control loops.
- Goal management: ques and priorities, objective setting, self-reflection module to check progress (can be other LLM agents).
- Verification & guardrails: fact checking modules, safety filters, external (LLMs) critiques.
- User/system interface: conversational (chat, voice, ...).

Questions?



Maastricht University

Exercise 1: Pre-Training Parameter Budget and Compute Scheduling (BERT-Base)

Review the parameter count derivation and pre-training regime for BERT-Base ($L = 12$ layers, hidden dimension $d = 768$, $h = 12$ attention heads, vocabulary size $V = 30,000$, and maximum sequence length $P = 512$).

- Calculate the parameter counts for the embedding layer components: Token Embeddings ($V \times d$), Positional Embeddings ($P \times d$), and Segment Embeddings ($2 \times d$). Explain why all three components must remain fully trainable rather than freezing the token embeddings.
- For a single Transformer layer, show how the parameter count is calculated across:
 - Multi-Head Self-Attention: Query, Key, Value projections ($3 \times d^2$) and the output projection (d^2).
 - Intermediate Feed-Forward Network: the expansion to $d_{\text{ff}} = 3072$ ($d \times 3072$) and projection back ($3072 \times d$).
 - LayerNorm parameters.
 - Combine these to verify that each layer contains approximately 7.08M weights, resulting in ~ 108.1 M total trainable parameters across 12 layers.
- Using the pre-training schedule from the lecture:

$$\text{Epochs} = \frac{\text{Steps} \times \text{Batch Size} \times \text{Sequence Length}}{\text{Total Tokens in Corpus}}$$

- Given a corpus of 3.63 billion tokens, a batch size of 256 sequences, and a sequence length of 512 tokens, calculate the number of steps required to complete 1 epoch and verify why 1,000,000 steps equates to approximately 40 epochs. Why are only 5--10 epochs recommended when performing domain-specific continual pre-training (e.g., BioBERT or LegalBERT)?

Exercise 2: Alignment via RLHF (SFT, Reward Modeling, and PPO)

The lecture details the three-stage Reinforcement Learning from Human Feedback (RLHF) framework used to align models like InstructGPT and ChatGPT.

- In Step 1 (Supervised Fine-Tuning - SFT), why does the model begin overfitting on validation loss after just 1 epoch, yet continuing to train for 16 epochs still improves downstream human preference ratings and reward model scores?
- In Step 2 (Reward Model - RM training), labelers rank $K \in [4,9]$ model completions per prompt, producing $\binom{K}{2}$ comparison pairs. Explain why training a smaller 6B parameter model as the reward function was preferred over using a 175B parameter model.
- In Step 3 (PPO), the objective function optimizes policy responses against the reward model:
 - How is the environment formulated as a multi-armed bandit problem?
 - Why is a per-token Kullback-Leibler (KL) divergence penalty between the active RL policy and the initial SFT model incorporated into the objective function? What failure mode (e.g., reward hacking or policy collapse) does the KL penalty prevent?



Exercise 3: Test-Time Reasoning and Reward Supervision (Outcome vs. Process Supervision)

The lecture explores how large language models transition from surface prompting to structured problem solving using Chain-of-Thought (CoT), Tree-of-Thought (ToT), and Process Reward Models (PRMs).

- Contrast **Chain-of-Thought with Self-Consistency (CoT-SC)** and **Tree-of-Thought (ToT)**:
 - How does CoT-SC use Monte Carlo sampling over diverse reasoning paths and majority voting to approximate a Bayesian Maximum A Posteriori (MAP) solution?
 - How does ToT utilize breadth-first search (BFS) or depth-first search (DFS) alongside state evaluation (voting/value scoring) to solve combinatorial search problems like the "Problem of 24"?
- Explain the fundamental flaw of an **Outcome-based Reward Model (ORM)** when applied to complex mathematical derivations (the credit-assignment problem).
- In Lightman et al. (OpenAI, 2023; PRM800K), how does a **Process-Supervised Reward Model (PRM)** score intermediate reasoning steps? Explain how test-time compute can be scaled via "Best-of- N " rejection sampling to boost accuracy on benchmarks like MATH without executing full tree search.

Discussion



Maastricht University

On Process Supervision vs. Outcome Supervision: Can a Model Learn Without Understanding Why It Failed?

In traditional reinforcement learning from human feedback (RLHF), an outcome-based reward model assigns a single scalar reward at the very end based on whether the final answer is right or wrong.

Yet the lecture out the classic credit-assignment problem: when a multi-step mathematical derivation arrives at the wrong conclusion, an outcome reward cannot identify which specific step contained the fatal flaw, nor can it reward sound intermediate logic. OpenAI's PRM800K work addresses this by scoring every newline step individually using process-supervised reward models.

But if human experts must manually annotate 800,000 intermediate reasoning steps with labels like correct, incorrect, or neutral, are we actually teaching the neural network to reason autonomously, or are we simply shifting the burden of brute-force human supervision deeper into the model's internal thought chain?

At what point does supervising the process stop scaling? What role can pure-play RL offer? What do we then need?



On the Two Scaling Axes: Parrots vs. Maze Rats

The lecture quotes a provocative comparison from Lightman et al. and Bender et al.:
'There are now two axes on which we can scale: training time and run time, monkeys and years, parrots and rats'.

For years, scaling meant pre-training bigger models on more trillions of web tokens—the 'stochastic parrot' paradigm.

Now, models like OpenAI's o-series scale test-time compute through search and sampling (like Monte Carlo Tree Search and Best-of- N reranking), acting like 'maze rats' exploring paths in a decision tree before answering.

If an LLM is still fundamentally an autoregressive token predictor at its core, does wrapping it in test-time search loops and self-consistency voting bridge the gap to genuine deductive thought?

Or are we just building faster mazes for the same stochastic parrot?



On the 'Brain in a Jar' Metaphor: Does an Embodied Wrapper Fix the Core Flaw?

The lecture describes a bare foundation model as a 'brain in a jar'—a text generator with no real grounding, no live world model, and no agency.

To create an AI agent, we wrap that jar with external vector databases for memory, API function calls for bodies and hands, and planning loops for goals.

However, the lectures also reminds us that an Atari 2600 running a 1979 program can defeat modern frontier LLMs at chess because the Atari maintains an exact, verifiable world state in memory, whereas LLMs make predictions over text tokens.

If the central predictive engine inside the agent still lacks an innate, grounded internal world model, does giving it external tools and memory actually make it an autonomous agent, or does it simply amplify the blast radius when the underlying text generator hallucinates?